P390

Britt Anderson

2025-01-01

Table of contents

Preface

This course is an effort to expose research minded undergraduate students in psychology to a variety of computational and programming tools that they can use to ease their research tasks. It is also intended to provide them practice at using such tools. Although I hope the particular tools that I choose for this are useful now, they may not be in the future. Things change fast in this area: smartphones are less than twenty years old, and practically useful chatbots are less than five years old. The emphasis is therefore on learning how to teach yourself to learn to use useful computer based tools.

These notes are written as a series of `qmd` files. This makes it a Quarto book. Quarto can be seen as a successor to RStudio and `Rmd` files. But rather than focusing on Quarto or `qmd` files per se, what I hope people will take away is that plain text files are enduring, human readable, and easily shareable. Computers can help with all the complicated syntax, and that is where **markdown** variants are useful. `qmd` files are one such variant, and Quarto is the tool that takes our plain text and turns it into something pretty. First, we wrote in $_{\text{notepad}}$, then we used RStudio, and now we are using Quarto (which you can use in RStudio, but which we will use from inside VSCode). What will you do when Quarto is passé? Or a collaborator wants to use a different markdown variant? You need to be able to understand how to change your syntax, download and install the new tool. That meta-knowledge should be your real goal in this course.

The subsequent material is generally ordered in the way I intend to teach it. These notes are not self-contained, but are intended to be reference material. If I mention a tool in class this is where you can find the name and link if you did not catch it. I may also demonstrate tool use in these pages and there will often be code that you can cut and paste to try things out. Think of this book more as a "student's guide" than as a textbook.

Chapter 1

Introduction

This is a book being created for PSYCH390 at the the University of Waterloo.

It is very much a work in progress. You can help it get better by providing feedback. Ideally you will do this by raising an issue on the github repo. Even better is if you would fix the book yourself and make a pull request.

Chapter 2

How can a computer help us in our research?

Class Question

What are steps (or stages) involved in psychological research?

2.1 Open Research Practices

Class Question

- What is "open research"?
- Should research be open?

10 strategies for open science

Class Question

What are the **Fair** principles for research?

- Fair principles
- Fair guiding principles

Read and discuss: A toolkit for transparency ….

Chapter 3

Basic Tools for Scientific Computing

In the following some of the common tools are highlighted with instructions for how to download and install.

3.1 Integrated Development Environments (IDEs)

- Who are you writing code for? Human or Machine?
- What is an IDE? What makes for a good IDE?

3.1.1 Required for this Course: VS Code

VSCode Opensource alternative

3.1.1.1 Installing VS Code

`See the operating system specific appendices.`

3.1.1.2 Using VSCode

https://www.youtube.com/watch?v=B-s71n0dHUk

- Basics video
- Using VSCode with R It is important that you do this for the operating system you are working with. Your VS Code environment will need to be able to "find" the R program when it wants to use it. Thus, when the instructions say "install R" you have to figure out if you are installing for WSL or Windows. For OSX and Linux native it is comparatively easier,

because there will be only one installation. For WSL you will want to follow the installation instructions for ubuntu making sure you execute these in your linux terminal (not power shell). If you have questions about adding things to your package database ask your instructor or TA.

- Using VSCode with Python The same caveats apply for Windows users. You want to be able to see things on the WSL Linux terminal side. You will want to install `uv` (per the instructions in the appendix) on the linux terminal side. When using VS Code (hopefully launched from your linux terminal) you will need to make sure that your VS Code app is using the correct virtual environment. This can be tricky (and frustrating). Ask for assistance if you have trouble with this.

3.1.2 Other Options

Emacs. VIM Jupyter Notebooks An environment for interactively analyzing and visualizing data. Can generate very nice output. It can be mixed with VS Code, but is most often used directly in a web browser. https://www.youtube.com/watch?v=suAkMeWJ1yE

3.2 Quarto

Quarto is a scientific and technical publishing system that allows you to combine text, code, and output into a single document.

3.2.1 Setup for VS Code

1. **Install Quarto CLI**: Download and install from quarto.org. Again, yet another warning for Windows users. You may also need to install this in the linux terminal so that you have it available on the linux side of things. Currently, the easiest way to do this seems to be to download the "deb" file from here to a directory visible within the linux side (you can find that in your windows finder. Scroll down to network and you can open up the linux, ubuntu, home, your user name, and place it there if you, for example, downloaded it to the windows download folder). Then you will need to `cd` your linux terminal to the same exact directory and run a particular command: `sudo dpkg -i <name of quarto deb file>`.
2. **VS Code Extension**: Install the **Quarto** extension from the VS Code Marketplace.

After almost all these new installations you will want to close and restart your VS Code index so that all the paths are refreshed and it can find all the new languages and extensions you have installed.

3.2.2 Your First Document

Create a new file named `test.qmd`.

At the very top of the file, you need a YAML header. This tells Quarto how to process the document. It sits between two lines of three dashes.

Type the following at the top of your file:

```
---
title: "My First Quarto Doc"
format: html
---
```

Below the header, you can write normal text. Add a section header and some text:

3.3 Introduction

This is a Quarto document. It combines markdown text with code.

Now let's add some math. Quarto uses LaTeX syntax for equations.

3.4 Math Example

We can write mathematical equations using LaTeX syntax.

$$f(x) = \int_{-\infty}^{\infty} \hat{f}(\xi)\, e^{2\pi i \xi x}\, d\xi$$

Inline math is also supported: $e^{i\pi} + 1 = 0$.

3.4.1 Previewing and Rendering

- **Preview**: Open the Command Palette (`Ctrl+Shift+P` or `Cmd+Shift+P`) and type `Quarto: Preview`. This will open a live preview window that updates as you edit.
- **Render**: Click the **Render** button at the top of the editor to generate the final output (HTML, PDF, etc.).

3.5 Installing R and Python

See the OS specific instructions:

- Mac Installation Instructions
- Windows Installation Instructions

3.5.1 Testing Your Installation of Python and R

Once you have installed the necessary tools, test your setup with the following exercises.

3.5.1.1 Python Test

Create a file named `test_python.py` and add the following code:

```python
def greet(name):
    """A simple function that greets someone by name."""
    return f"Hello, {name}! Welcome to scientific computing."

# TODO: Write your own function below
# Create a function called 'square' that takes a number and returns its square
```

To test your work: 1. Open a Python interactive session (in VS Code or your terminal) 2. Import your file: `import test_python` 3. Test the provided function: `test_python.greet("Student")` 4. Test your function: `test_python.square(5)` (should return 25)

3.5.1.2 R Test

Create a file named `test_r.R` and add the following code:

```r
# A simple function that greets someone by name
greet <- function(name) {
  paste("Hello", name, "! Welcome to scientific computing.")
}

# TODO: Write your own function below
# Create a function called 'square' that takes a number and returns its square
```

To test your work: 1. Open an R interactive session (in VS Code or RStudio) 2. Source your file: `source("test_r.R")` 3. Test the provided function: `greet("Student")` 4. Test your function: `square(5)` (should return 25)

3.5.2 Submission

Once you've completed both exercises and verified they work in an interactive session, save your files. This confirms your development environment is properly configured. Your test files will need to be submitted on Learn for grading.

Chapter 4

Terminals

Currently these words are used as synonyms: terminal, shell, command line; which is not exactly true, but is close enough.

https://vimeo.com/453837142

- Power Shell.
- OSX Terminal

4.1 Commands to try (many have options)

- `ls`
- `ls -alt`
- `cd`
- `pwd`
- `mkdir`
- `rm` **DANGER**
- `mv`
- `cp`
- `cat`
- `less`
- `find` **very powerful**
- `du`
- `df`
- `touch`
- `ln`
- `chmod`
- `chown`

4.2 Additional Introductory Material

1. The command line
2. A Short Series of Terminal Lessons from the UbuntuWiki
3. Some Scripting Basics
4. Another Scripting Introduction

4.3 Getting work done: scripting

Want to convert and compress a large directory of videos (as I did for the pandemic version of a course)?

```
for i in *.MP4;
do name=`echo "$i" | cut -d'.' -f1`;
echo "$name";
ffmpeg -i "$i" -c:v libx264 -b:v 1.5M -c:a aac -b:a 128k "${name}S.mp4";
done
```

4.4 Talking to other computers in the terminal

4.4.1 SSH

Testing and Learning for Linux SSH

4.5 Terminal Self-Learning Exercises for Psychology Students

4.5.1 Prerequisites

- You have located your terminal application (Terminal on Mac, WSL terminal on Windows)
- You can open it successfully

4.5.2 Note on Command Compatibility

These exercises work in both bash (WSL/Linux) and zsh (Mac). The commands are identical between these shells. To avoid conflicts you should use the `#!/bin/sh` at the start of the script. That should work for basic functionality on both systems.

4.5.3 Exercise Set 1: Orientation and Basic Navigation

4.5.3.1 Learning Goals

- Understand what the prompt means
- Know where you are in the file system
- Navigate between directories

4.5.3.2 1.1 Where Am I?

Open your terminal. You'll see a prompt (something like `user@computer:~$` or `computer-name:~ user$`).

Task: Type `pwd` and press Enter.

Question to answer: What does `pwd` stand for? What information does it give you?

Expected outcome

You should see a path like `/home/username` (Linux/WSL) or `/Users/username` (Mac). This is your "home directory" - your starting location.

`pwd` = "print working directory"

4.5.3.3 1.2 What's Here?

Task: Type `ls` and press Enter.

Question to answer: What does this command show you?

Expected outcome

You see a list of files and folders in your current directory. Common ones might include Documents, Downloads, Desktop.

`ls` = "list"

4.5.3.4 1.3 More Details

Task: Type `ls -l` and press Enter.

Question to answer: How is the output different from plain `ls`? What additional information do you see?

Expected outcome

You see the same files but with permissions, owner, size, and modification date. The `-l` is called a "flag" or "option" that modifies command behavior.

4.5.3.5 1.4 Including Hidden Files

Task: Type `ls -la` and press Enter.

Question to answer: Do you see any files starting with `.` (period)? What do you think these are?

Expected outcome

Files starting with `.` are hidden by default. They're often configuration files like `.bashrc` or `.zshrc`.

4.5.3.6 1.5 Moving Around

Task: 1. Type `cd Documents` and press Enter 2. Type `pwd` to see where you are now 3. Type `ls` to see what's in Documents

Question to answer: Did your prompt change? How can you tell you're in a different location?

Expected outcome

Your working directory changed to `/home/username/Documents` (or similar). The prompt might show the directory name.

`cd` = "change directory"

4.5.3.7 1.6 Going Back

Task: Type `cd ..` and press Enter, then `pwd`.

Question to answer: Where are you now? What does `..` mean?

Expected outcome

You're back in your home directory. `..` means "parent directory" (one level up).

4.5.3.8 1.7 Returning Home

Task: 1. Navigate into Documents again: `cd Documents` 2. Type `cd` (with nothing after it) and press Enter 3. Type `pwd` to verify

Question to answer: Where does `cd` by itself take you?

Expected outcome

Plain `cd` always returns you to your home directory, no matter where you are.

4.5.4 Exercise Set 2: Creating and Manipulating Files

4.5.4.1 Learning Goals

- Create directories and files

- Move and copy things
- Delete safely

4.5.4.2 2.1 Make a Workspace

Task: 1. Make sure you're in your home directory: `cd` 2. Type `mkdir terminal_practice` and press Enter 3. Type `ls` to verify it was created 4. Type `cd terminal_practice`

Question to answer: What do you think `mkdir` stands for?

Expected outcome

You created a new directory and moved into it.

`mkdir` = "make directory"

4.5.4.3 2.2 Create a File

Task: Type `touch experiment_notes.txt` and press Enter, then `ls`.

Question to answer: Does a file appear? Open your graphical file browser (Finder/Explorer) and navigate to this folder. Can you see the file there?

Expected outcome

You created an empty file. `touch` creates files (technically, it updates timestamps, but creates the file if it doesn't exist).

4.5.4.4 2.3 Add Content to a File

Task: Type `echo "First observation: Subjects prefer terminal over GUI" > experiment_notes.txt` and press Enter.

Question to answer: What do you think `>` does? Try opening the file in a text editor to check.

Expected outcome

The text was written to the file. `>` means "redirect output to file" (it overwrites existing content).

4.5.4.5 2.4 Append to a File

Task: 1. Type `echo "Second observation: Learning curve steep" >> experiment_notes.txt` 2. View the file content: `cat experiment_notes.txt`

Question to answer: How is `>>` different from `>`?

Expected outcome

You see both lines. `>>` appends (adds to the end) instead of overwriting.

`cat` = "concatenate" (displays file contents)

4.5.4.6 2.5 Copy Files

Task: 1. Type `cp experiment_notes.txt backup_notes.txt` 2. Type `ls` to verify both files exist

Question to answer: What do you think `cp` stands for?

Expected outcome

You now have two identical files.

`cp` = "copy" Format: `cp source destination`

4.5.4.7 2.6 Rename/Move Files

Task: Type `mv backup_notes.txt observations.txt` and then `ls`.

Question to answer: What happened to `backup_notes.txt`?

Expected outcome

The file was renamed. `mv` = "move" but is also used for renaming (moving to a new name in the same directory).

4.5.4.8 2.7 Create Directory Structure

Task: 1. Type `mkdir data` 2. Type `mkdir data/raw data/processed` 3. Type `ls data`

Question to answer: Can you create multiple directories at once?

Expected outcome

You created a data directory with two subdirectories. You can specify multiple paths to mkdir.

4.5.4.9 2.8 Move Files Between Directories

Task: 1. Create a test file: `touch subject_001_data.csv` 2. Move it: `mv subject_001_data.csv data/raw/` 3. Verify: `ls data/raw`

Question to answer: How do you move files between directories?

Expected outcome

The file is now in data/raw/. When moving, you specify the destination directory.

4.5.4.10 2.9 Safe Deletion

Task: 1. Type `rm observations.txt` 2. Type `ls` to verify it's gone 3. Try `ls observations.txt`

Question to answer: Can you get the file back after using `rm`?

Expected outcome

The file is permanently deleted. There's no "recycle bin" in the terminal. **Be careful with `rm`!**

`rm` = "remove"

4.5.4.11 2.10 Remove Directories

Task: 1. Try `rm data` 2. Now try `rm -r data`

Question to answer: Why doesn't the first command work? What does the `-r` flag do?

Expected outcome

Plain `rm` doesn't work on directories. The `-r` flag means "recursive" (remove directory and everything inside).

Warning: `rm -r` is dangerous - it deletes everything in the directory. Always double-check before using it.

4.5.5 Exercise Set 3: Viewing and Searching Content

4.5.5.1 Learning Goals

- Read file contents in different ways
- Search within files
- Understand pipes and filters

4.5.5.2 3.1 Setup the Dataset

Task: Let's create a sample dataset for practice.

```
cd ~/terminal_practice
cat > participants.txt << 'EOF'
ID,Age,Condition,Score
001,23,Control,78
002,27,Treatment,85
003,21,Control,72
004,25,Treatment,91
005,29,Control,68
006,22,Treatment,88
007,26,Control,75
008,24,Treatment,82
EOF
```

Just copy all of this and paste it into your terminal, then press Enter.

Question to answer: Type `cat participants.txt` - do you see your data?

Expected outcome

You created a CSV file with participant data. The `<< 'EOF'` syntax is called a "here document" - a way to input multiple lines.

4.5.5.3 3.2 Page Through Files

Task: 1. Type `less participants.txt` and press Enter 2. Use arrow keys to navigate 3. Press `q` to quit

Question to answer: How is `less` different from `cat`?

Expected outcome

`less` allows you to scroll through files without printing everything to the screen. Useful for large files.

Navigation: arrows to move, `q` to quit, `/` to search

4.5.5.4 3.3 First Few Lines

Task: Type `head participants.txt`

Question to answer: How many lines does it show?

Expected outcome

Shows first 10 lines by default. Useful for checking file structure.

Try: `head -n 3 participants.txt` to show only 3 lines

4.5.5.5 3.4 Last Few Lines

Task: Type `tail participants.txt`

Question to answer: What do you see?

Expected outcome

Shows last 10 lines. Useful for checking end of files or recent log entries.

4.5.5.6 3.5 Count Lines

Task: Type `wc -l participants.txt`

Question to answer: What number do you get? Why?

Expected outcome

You should see `8 participants.txt` (or possibly 9 depending on blank lines).

`wc` = "word count", `-l` flag means count lines instead

4.5.5.7 3.6 Search for Patterns

Task: Type `grep Treatment participants.txt`

Question to answer: What gets displayed?

Expected outcome

Only lines containing "Treatment" are shown. `grep` searches for patterns in files.

4.5.5.8 3.7 Search with Count

Task: Type `grep -c Treatment participants.txt`

Question to answer: What does this number represent?

Expected outcome

The count of lines containing "Treatment". The `-c` flag gives you a count instead of the lines themselves.

4.5.5.9 3.8 Case-Insensitive Search

Task: 1. Type `grep control participants.txt` 2. Type `grep -i control participants.txt`

Question to answer: Why different results?

Expected outcome

First search finds nothing (lowercase doesn't match "Control"). Second search with `-i` flag is case-insensitive.

4.5.5.10 3.9 Combining Commands (Pipes)

Task: Type `cat participants.txt | grep Treatment | wc -l`

Question to answer: What happened? What does the `|` symbol do?

Expected outcome

This counts Treatment participants. The `|` (pipe) sends output of one command as input to the next.

Flow: display file → filter for Treatment → count lines

4.5.5.11 3.10 Sorting Data

Task: 1. Type `sort participants.txt` 2. Type `sort -k4 -t, -n participants.txt`

Question to answer: How does the second command change the sort?

Expected outcome

First sorts alphabetically by whole line. Second sorts numerically (`-n`) by field 4 (`-k4`) using comma as delimiter (`-t,`) - i.e., by Score column.

4.5.6 Exercise Set 4: Efficiency Tools

4.5.6.1 Learning Goals

- Use command history
- Autocomplete with Tab
- Edit commands efficiently
- Use wildcards

4.5.6.2 4.1 Command History

Task: 1. Type `history` and press Enter 2. Use up-arrow key a few times 3. Press down-arrow

Question to answer: What do the arrow keys do?

Expected outcome

`history` shows all past commands. Up/down arrows cycle through command history - very useful for repeating or modifying previous commands.

4.5.6.3 4.2 Search History

Task: 1. Press `Ctrl+r` (Control-r) 2. Start typing `grep` 3. Press `Ctrl+r` again to see next match

Question to answer: What does `Ctrl+r` do?

Expected outcome

Reverse search through command history. Start typing and it finds matching commands. Press Enter to run, or edit first.

4.5.6.4 4.3 Tab Completion

Task: 1. Type `cd ~/term` and press Tab 2. If it doesn't complete, press Tab twice

Question to answer: What happened?

Expected outcome

Tab autocompletes file/directory names. Double-tab shows all matches if there are multiple options.

This saves enormous amounts of typing and prevents typos.

4.5.6.5 4.4 Cursor Movement

Task: Type a long command (don't press Enter yet): `echo "This is a very long command that I want to edit"`

Now practice: - `Ctrl+a` (jump to start) - `Ctrl+e` (jump to end) - `Alt+b` or `Esc` then `b` (back one word) - `Alt+f` or `Esc` then `f` (forward one word)

Question to answer: Are these faster than arrow keys?

Expected outcome

These shortcuts make editing commands much faster than holding arrow keys.

4.5.6.6 4.5 Wildcards - Multiple Files

Task: 1. Create test files: `touch data_jan.csv data_feb.csv data_mar.csv report_jan.txt` 2. Type `ls data*` 3. Type `ls *.csv` 4. Type `ls data_*.csv`

Question to answer: What does `*` match?

Expected outcome

`*` is a wildcard matching any characters. - `data*` matches anything starting with "data" - `*.csv` matches anything ending with ".csv"
- `data_*.csv` matches that pattern

This is powerful for operating on multiple files at once.

4.5.6.7 4.6 Wildcards - Single Character

Task: 1. Type `ls data_???.csv`

Question to answer: How is `?` different from `*`?

Expected outcome

`?` matches exactly one character. So `???` matches exactly three characters (jan, feb, mar).

4.5.6.8 4.7 Clear the Screen

Task: 1. Your terminal is probably cluttered now. Type `clear` 2. Alternatively, press `Ctrl+l`

Question to answer: Does this delete your history?

Expected outcome

Screen is cleared but history remains. You can still scroll up or use up-arrow to access previous commands.

4.5.7 Exercise Set 5: Practical Research Workflow

4.5.7.1 Learning Goals

- Combine tools for real tasks
- Develop a reproducible workflow
- Practice documentation

4.5.7.2 5.1 Organize a Project

Task: Create this directory structure:

```
cd ~
mkdir psych_study
cd psych_study
mkdir data data/raw data/processed scripts results
```

Then verify: `ls -R`

Question to answer: What does `ls -R` do?

Expected outcome

You created a standard research project structure. `-R` shows recursive listing (shows subdirectories too).

4.5.7.3 5.2 Create a Data Log

The task is illustrated with the use of `echo`. This will work and requires no additional tools to get familiar with. However, it can be tedious. Most systems will come with a very minimalist word processor built in. You might find that more convenient and you are welcome to use it if you prefer. A very common minimal text editor that launches in the terminal is "nano". To see if you have this just type `nano` at the prompt. The "^" character in the menu displayed at the problem means your control (aka CTRL) key. To save a file for example you would type `Ctrl-o` at the same time. Then you would enter the name of the file and hit enter to save. **Task:**

```
cd ~/psych_study
echo "# Data Collection Log" > README.txt
echo "" >> README.txt
echo "Date: $(date)" >> README.txt
echo "Researcher: [Your Name]" >> README.txt
echo "Study: Terminal Skills Acquisition" >> README.txt
```

Then view it: `cat README.txt`

Question to answer: What does `$(date)` do?

Expected outcome

`$(date)` runs the date command and inserts its output. This is called "command substitution" - very useful in scripts.

4.5.7.4 5.3 Document Your Commands

Task: Create a script file:

```
cd ~/psych_study/scripts
touch process_data.sh
echo '#!/bin/sh' > process_data.sh
echo "# Script to process participant data" >> process_data.sh
echo "" >> process_data.sh
echo "echo 'Starting data processing...'" >> process_data.sh
echo "ls ../data/raw/" >> process_data.sh
echo "echo 'Processing complete!'" >> process_data.sh
```

View it: `cat process_data.sh`

Question to answer: What is `#!/bin/sh` at the top?

Expected outcome

This is called a "shebang" - it tells the system this is a shell script. Scripts are just text files containing commands.

4.5.7.5 5.4 Make Scripts Executable

Task: 1. Try running: `./process_data.sh` 2. It fails! Now type: `chmod +x process_data.sh` 3. Try again: `./process_data.sh`

Question to answer: What did `chmod +x` do?

Expected outcome

`chmod +x` makes the file executable. Without it, the system won't run it.

`chmod` = "change mode" (permissions)

4.5.7.6 5.5 Batch Process Files

Task: Create sample data files and process them:

```
cd ~/psych_study/data/raw
echo "subject,rt,accuracy" > pilot_01.csv
echo "S001,450,1" >> pilot_01.csv
echo "S002,523,1" >> pilot_01.csv

echo "subject,rt,accuracy" > pilot_02.csv
echo "S003,489,0" >> pilot_02.csv
echo "S004,512,1" >> pilot_02.csv
```

```
# Now count how many data rows across all files
cat pilot_*.csv | grep -v "subject" | wc -l
```

Question to answer: What does `grep -v` do?

Expected outcome

You counted total participants across multiple files. `grep -v` inverts the match (shows lines NOT containing the pattern) - here, excluding headers.

4.5.7.7 5.6 Extract Specific Data

Task:

```
# Extract all accurate responses (accuracy=1)
grep ",1$" pilot_*.csv
```

Question to answer: What does `$` mean in the pattern?

Expected outcome

`$` means "end of line" in grep patterns. So `,1$` matches lines ending with ",1".

This is a "regular expression" - a powerful pattern matching system.

4.5.7.8 5.7 Create Summary Statistics

Task:

```
cd ~/psych_study/results
echo "# Summary Statistics" > summary.txt
echo "" >> summary.txt
echo "Total data files: $(ls ../data/raw/*.csv | wc -l)" >> summary.txt
echo "Total participants: $(cat ../data/raw/*.csv | grep -v subject | wc -l)" >> summa
cat summary.txt
```

Question to answer: Did this create a useful summary?

Expected outcome

You automated summary statistics. This approach scales - if you add 100 more files, the command still works.

4.5.7.9 5.8 Find Files by Content

Task:

```
cd ~/psych_study
grep -r "S001" .
```

Question to answer: What does `-r` do with grep?

Expected outcome

`-r` makes grep recursive - it searches in current directory and all subdirectories. Useful for finding where specific data appears.

4.5.7.10 5.9 Check Disk Usage

Task:

```
cd ~/psych_study
du -sh *
```

Question to answer: What does this tell you?

Expected outcome

`du` = "disk usage" `-s` = summary for each item `-h` = human-readable (KB, MB instead of bytes)

Shows how much space each directory uses - important for large datasets.

4.5.7.11 5.10 Final Cleanup Exercise

Task: Time to clean up our practice.

```
cd ~
ls terminal_practice  # Verify it exists
rm -r terminal_practice  # Remove it

ls psych_study  # Verify it exists
rm -r psych_study  # Remove it
```

Question to answer: Why is it important to be careful with `rm -r`?

Expected outcome

You successfully cleaned up practice directories. Always double-check paths before using `rm -r` - there's no undo!

4.5.8 Self-Assessment Checklist

After completing these exercises, you should be able to:

- ☐ Navigate the file system (`cd`, `pwd`, `ls`)
- ☐ Create and remove files and directories (`touch`, `mkdir`, `rm`, `rm -r`)
- ☐ Copy and move files (`cp`, `mv`)
- ☐ View file contents (`cat`, `less`, `head`, `tail`)
- ☐ Search within files (`grep`)
- ☐ Count and sort data (`wc`, `sort`)
- ☐ Use pipes to combine commands (`|`)
- ☐ Use wildcards for batch operations (`*`, `?`)
- ☐ Navigate command history (arrow keys, `Ctrl+r`)

- ☐ Use tab completion
- ☐ Create simple shell scripts
- ☐ Make files executable (`chmod +x`)

4.5.9 Common Pitfalls and Tips

4.5.9.1 Safety

- **Always** check `pwd` before running `rm -r`
- There is no "Undo" in the terminal
- Test commands on unimportant files first

4.5.9.2 Efficiency

- Use Tab completion religiously - prevents typos
- Use up-arrow for recent commands instead of retyping
- Learn `Ctrl+r` for searching history - it's incredibly useful

4.5.9.3 Debugging

- If a command doesn't work, check for typos (especially spaces and `-` vs `_`)
- File names with spaces need quotes: `"my file.txt"` or escaping: `my\ file.txt`
- Case matters: `File.txt` ≠ `file.txt`

4.5.9.4 Getting Help

- `man <command>` shows the manual (press `q` to quit)
- `<command> --help` usually shows brief help
- Google "bash [command] examples" for more information

4.5.10 Next Steps

Once comfortable with these basics, you might explore: - **Text processing**: `awk`, `sed` for data manipulation - **Version control**: `git` for tracking changes - **Remote access**: `ssh` for working on servers - **Scripting**: Writing longer bash/zsh scripts for automation - **Package managers**: `brew` (Mac) or `apt` (Linux) for installing software

Remember: The terminal seems arcane at first, but it's actually more logical than most GUIs once you understand the patterns. These commands have remained largely unchanged for 40+ years because they work well.

4.5.11 Getting Stuck?

If you get stuck or your terminal stops responding: - `Ctrl+c` cancels the current command - `Ctrl+d` exits many programs
- If completely stuck, close the terminal window and open a new one - Type `reset` if your terminal looks weird/corrupted

---

Chapter 5

Version Control

5.1 Git

5.1.1 Git, Github, and Version Control Background

5.1.1.1 What is Git and GitHub?

Git is a version control system that tracks changes to files over time. Think of it as an incredibly sophisticated "undo" system that also lets multiple people work on the same files.

GitHub is a website that hosts git repositories online. It's like Google Drive for code, but with powerful collaboration features built specifically for version control.

5.1.1.2 Why Use Git?

- **Track changes**: See exactly what changed, when, and by whom
- **Collaborate**: Work with others without overwriting each other's work
- **Backup**: Your code lives both on your computer and on GitHub
- **Experiment safely**: Try new things without fear of breaking what works
- **Professional skill**: Git is the industry standard for version control

https://vimeo.com/456349738

https://vimeo.com/456349595

5.1.1.3 Git Is Not The Only Option: Other version control system

1. Mercurial
2. Darcs
3. CVS

4. Subversion
5. Pijul

5.1.1.4 Git is not Github

And github is not the only way to share code and data using `git` version control.

1. OSF.io
2. Sourceforge
3. Bitbucket
4. Gitlab The university provides you with a gitlab account: https://git.uwaterloo.ca
5. Codeberg

5.1.1.5 What Does Forking Mean? Some Git Terminology

- **Fork:** A copy of one github repository to another **github** repository.
- **Clone:** A copy of one **git** repository to another **git** repository.
- **Remote:** This is a repository that you are following. You will typically *pull* from these, but your *push* permissions may be limited depending on the distinctions between forks and clones, and whether you own the remote or someone else does. You can have more than one remote.
- **Pull:** the transfer of information and changes **from** one repository incorporated into another. This is how you get the new information from a remote transported to a local repository that you control.
- **Push:** this is the transfer of information **to** a repository you control (or have permissions to push to) from another repository that you control. This is often from your local laptop version to the hosted repository (your fork) on github.
- **Pull Request:** When you have information or changes that you think would be helpful to a remote you do **not** have push permissions for then you can request that the owner of that repository pull in your changes. This is a formal process called a pull request. It is primarily a github concept and not a git concept.
- **Branch:** within a repository the development of the code may be proceeding in a few different directions at the same time. The principal production branch is conventionally called **master**. And the principal repository that is the main, shared one is called **origin**. We will not be working with branches in our course, but those terms do show up in commands.

5.1.2 Getting Git And Checking Your Setup

It depends on what system you have. See the operating system appendices.

5.1.2.1 Check Your Git Installation

Task: Open your terminal and type: `git --version`

Question to answer: Do you see a version number (like "git version 2.x.x")?

Expected outcomes

If you see a version number: Git is already installed! Skip to section 2.3.

If you see "command not found": You need to install git (proceed to 2.2).

Troubleshooting

If installation fails: - Mac: You might need to install Xcode Command Line Tools first - WSL: Make sure you're running the commands in your WSL terminal, not PowerShell - Both: Try closing and reopening your terminal after installation

5.1.2.2 Configure Your Name

Task: Type these commands in your terminal, replacing with YOUR information:

```
git config --global user.name "Your Name"
git config --global user.email "your.email@uwaterloo.ca"
git config --global pull.rebase true
```

Question to answer: Type `git config --global --list`. Do you see your name and email?

Why this matters

Every change you make in git will be labeled with this name and email. This is how collaborators know who made each change.

Use your real name (or professional pseudonym) and a real email - this information appears in the public commit history.

The `--global` flag means this configuration applies to all your git projects.

5.1.2.3 Set Your Default Branch Name

Task:

```
git config --global init.defaultBranch main
```

Question to answer: Why "main" instead of other names?

Expected outcome

"main" is the modern standard default branch name (replacing the older "master"). This ensures consistency with GitHub and current best practices.

A branch is a line of development. We'll learn more about branches later.

5.1.2.4 Verify Your Configuration

Task:

```
git config --global --list
```

Question to answer: Can you see all settings (user.name, user.email, init.defaultBranch, pull.rebase)?

Expected output

You should see something like:

```
user.name=Jane Smith
user.email=jane.smith@uwaterloo.ca
init.defaultBranch=main
pull.rebase=true
```

That last is intended to help deal with a particular error that cropped up anew for the class in 2026 when git changed some of its default behavior. Please let me know when, later on in the homework, if you see a "divergent branch message" **and** it is preventing you from pulling from my remote and pushing to your fork (terminology that should make more sense shortly).

These are stored in a hidden file called `~/.gitconfig`. You can edit this file directly if needed.

5.1.3 Communicating with Github

Communication will be much easier with an **ssh key**. And the easiest way to get an SSH key is via the terminal. If you haven't done any of the terminal exercises, now might be a good time to go there. Do at least the basics so you know where your terminal is, and then come back here to finish the set up.

5.1.3.1 Create a Github Account

Task:

1. Go to https://github.com
2. Click "Sign up"
3. Choose a professional username (you'll use this for your career)
4. Use your UWaterloo email if you're a student (you may get educational benefits)

Question to answer: What username did you choose? Will it be appropriate to share with future employers or graduate programs?

Best practices

Choose something professional:

- Good: `jane-smith`, `j-smith-research`, `jasmith`
- Avoid: `party-animal-2024`, `psycho-killer`, `coolguy123`

Your GitHub profile is essentially your coding resume. Many employers and grad programs will look at it.

5.1.3.2 Creating an ssh key with your terminal

Using SSH keys is the preferred method for authenticating with GitHub. It saves you from typing your username and password every time you push code.

Step 1: Generate the key pair

Open your terminal and run the following command (substituting your email):

```
ssh-keygen -t ed25519 -C "your.email@uwaterloo.ca"
```

1. When asked where to save the key, just press Enter to accept the default location (usually `~/.ssh/id_ed25519`).
2. When asked for a passphrase, you can press Enter for no passphrase (easier) or type a password for extra security.

Step 2: Copy the public key

You need to copy the contents of the public key file you just created.

Run this command to display the key:

```
cat ~/.ssh/id_ed25519.pub
```

Select and copy the entire output string (it starts with `ssh-ed25519` and ends with your email).

Step 3: Add the key to GitHub

1. Go to GitHub.com and sign in.
2. Click your profile photo in the top-right corner and select **Settings**.
3. In the "Access" section of the sidebar, click **SSH and GPG keys**.
4. Click the green **New SSH key** button.
5. **Title**: Give it a name like "My Laptop".
6. **Key type**: Leave as "Authentication Key".
7. **Key**: Paste the key you copied in Step 2.
8. Click **Add SSH key**.

Step 4: Test the connection

Back in your terminal, type:

```
ssh -T git@github.com
```

- If you see a warning about authenticity (`The authenticity of host...`), type `yes` and press Enter.

- You should see a success message: `Hi username! You've successfully authenticated...`

Important Note for Future Cloning:

Now that you have SSH set up, when you clone repositories, always select the **SSH** tab (instead of HTTPS) to get the URL. It will look like: `git@github.com:username/repository.git`

If you use the HTTPS URL, git will ask for your username/password instead of using your new key.

5.1.4 Using Git and Github — Practice

5.1.4.1 Fork the Course Repository

Task:

1. While viewing https://github.com/brittAnderson/uwlooP390
2. Click the "Fork" button in the upper right
3. Click "Create fork"
4. You should now see the repository under YOUR username: `https://github.com/YOUR-USERNAME/uwlooP390`

Question to answer: What does "forking" do?

Expected outcome

Forking creates your own personal copy of someone else's repository. You now have:

- **Original repo** (mine): `brittAnderson/uwlooP390` (you can't change this)
- **Your fork**: `YOUR-USERNAME/uwlooP390` (you can change this freely)

Your fork is completely independent - you can experiment without affecting the original.

5.1.4.2 More Forking

Task:

Repeat the forking process for https://github.com/brittAnderson/p390Practice

Question to answer: How many repositories do you now have in your GitHub account?

Expected outcome

You should have at least 2 repositories:

- `YOUR-USERNAME/uwlooP390` (forked from course repo)
- `YOUR-USERNAME/p390Practice` (forked from practice repo)

You can see all your repositories by clicking your profile icon → "Your repositories"

5.1.4.3 Explore Repository Features

Task: In your forked `p390Practice` repository, explore these tabs:

1. **Code** - the file browser
2. **Issues** - where people report problems or ask questions
3. **Pull requests** - where people submit changes
4. **Insights** → **Network** - visualize the repository's history

Question to answer: What do you see in the Network graph?

Expected observations

The Network graph shows:

- The original repository (mine)
- All forks (including yours)
- Branches and their relationships
- The flow of commits over time

This visualizes how code evolves and spreads through collaboration.

5.1.4.4 Take a Screenshot - You'll need this for homework

Task:

1. Navigate to your GitHub profile page (`https://github.com/YOUR-USERNAME`)
2. Take a screenshot showing your account with both forked repositories visible
3. Save it as `github_setup_complete.png`

Question to answer: Can you see both `uwlooP390` and `p390Practice` in your repository list?

Why this matters

This screenshot confirms you've completed the setup. You might submit this to show you're ready for collaborative work.

This is also your first step toward building a professional portfolio. As you add projects, your GitHub profile becomes evidence of your skills.

5.1.5 Cloning and Basic Git Operations

When you "fork" you are making a copy of a github repository from someone else's account to yours. Cloning happens either from github to your local machine (probably laptop) or from one local machine to another.

Note, in much of the following it assumes you are using the directory `git_workspace` and that it is in your home directory. But you may have put your files somewhere else, or used a different directory name. If so, you need to substitute those names and locations in what follows.

Task:

1. Make sure you're in your home directory: `cd ~`
2. Create a workspace: `mkdir git_workspace` and `cd git_workspace`

   It can be named anything. I generally put my repos in `~/gitRepos/`.
3. Go to YOUR fork of p390Practice on GitHub
4. Click the green "Code" button
5. Copy the HTTPS URL (if you did not create an ssh key ;should be `https://github.com/YOUR-USERNAME/p390Practice.git`). If you created an ssh key you should use the one that starts with `git@`.
6. In terminal: `git clone https://github.com/YOUR-USERNAME/p390Practice.git` . Again, this may be different if you successfully created your ssh key.
7. Type `ls` to verify, then `cd p390Practice`

You now have:

- **Remote** (on GitHub): `github.com/YOUR-USERNAME/p390Practice`
- **Local** (on your computer): `~/git_workspace/p390Practice`

Changes you make locally don't automatically appear on GitHub - you have to explicitly **push** them.

5.1.5.1 View Repository Contents

Task:

```
ls -la
```

Question to answer: Do you see a `.git` directory?

Expected observation

The `.git` directory (hidden by default) contains all of git's tracking information - the entire history, all branches, configuration, etc.

NEVER manually edit or delete the `.git` directory! This is where git stores everything.

Everything else in the folder is your "working directory" - the actual files you edit.

5.1.5.2 Check Repository Status

This is important. If multiple people are working on the same repository, which is very common, you will need to have all the latest changes from the principal repo first, before you push your local changes.

Task: This is how to do it in the terminal.

```
pwd  # Verify you're in ~/git_workspace/p390Practice
git status
```

Question to answer: What does `git status` tell you?

Expected output

You should see something like:

```
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

This means: - You're on the "main" branch - Your local copy matches what's on GitHub ("origin") - You haven't made any changes yet

5.1.6 Explore Remote Repositories — Your clone may have many

Remotes are what they sound like. Locations that are following the same repository that you are.

Task:

```
git remote -v
```

Question to answer: What does "origin" refer to?

Expected output

You should see:

```
origin  https://github.com/YOUR-USERNAME/p390Practice.git (fetch)
origin  https://github.com/YOUR-USERNAME/p390Practice.git (push)
```

"origin" is the default name for the remote repository you cloned from. In this case, origin is YOUR fork on GitHub.

We'll add a second remote (the original course repo) later.

5.1.6.1 Create a New File

Task:

```
mkdir my_practice
cd my_practice
touch learning_notes.txt
echo "Git is a version control system" > learning_notes.txt
cat learning_notes.txt
cd ..
```

Question to answer: Did git automatically track this new file?

Expected outcome

No! Creating a file doesn't automatically add it to git. Type `git status` and you'll see it listed under "Untracked files".

You have to explicitly tell git which files to track. This is a feature - it lets you have files in the directory that git ignores (like temporary files or local configurations).

5.1.6.2 Stage Your Changes (git add)

Task:

```
git status  # See that the directory is untracked
git add my_practice/learning_notes.txt
git status  # See the change
```

Question to answer: What changed when you ran `git add`?

Expected outcome

After `git add`, the file moves from "Untracked files" to "Changes to be committed" (shown in green).

This is called "staging" - you're telling git "I want to include this change in my next commit."

Think of staging as putting items in a shopping cart (you've selected them but haven't checked out yet).

5.1.6.3 Commit Your Changes

Task:

```
git commit -m "Add my practice directory with learning notes"
git status
```

Question to answer: What does committing do? What does the `-m` flag mean?

Expected outcome

After committing, `git status` should show "nothing to commit, working tree clean".

Commit = create a permanent snapshot of the staged changes.

`-m` lets you write the commit message directly in the command. The message should describe what changed and why.

Good commit messages:

- "Add learning notes file with git definition"
- "Fix typo in analysis script"
- "updated stuff"
- "asdf"

5.1.6.4 View Commit History

Task:

```
git log
```

Question to answer: Can you find your commit? What information does each commit show?

Expected output

Each commit shows:

- **Commit hash**: unique identifier (long string of letters/numbers)
- **Author**: name and email
- **Date**: when the commit was made
- **Message**: description of the change

Press `q` to exit the log viewer.

Try: `git log --oneline` for a compact view Try: `git log --graph --oneline --all` for a visual tree

5.1.6.5 Make More Changes

Task:

```
echo "GitHub is a hosting service for git repositories" >> my_practice/learning_notes.txt
cat my_practice/learning_notes.txt
git status
```

Question to answer: What does `git status` show now?

Expected outcome

Git detected that `learning_notes.txt` was modified. It shows under "Changes not staged for commit" (in red).

The file is tracked, but the new changes aren't staged yet. You need to `git add` again to stage the new changes, even though the file is already tracked.

5.1.6.6 The Add-Commit Cycle

Task:

```
git add my_practice/learning_notes.txt
git commit -m "Add GitHub definition to learning notes"
git log --oneline
```

Question to answer: How many commits do you have now?

Expected outcome

You should see your two new commits plus any that existed before.

This is the fundamental git workflow:

1. Make changes to files
2. `git add` to stage changes
3. `git commit` to create snapshot
4. Repeat

Each commit is a save point you can return to later.

5.1.6.7 Check Current Remotes

Task:

```
git remote -v
```

Question to answer: How many remotes do you have? What are they called?

Expected output

Right now you should only see "origin" (your fork on GitHub).

To collaborate effectively, we need to add a second remote: the original course repository (my repository that you forked from).

5.1.6.8 Add Upstream Remote

Task:

```
git remote add upstream https://github.com/brittAnderson/p390Practice.git
git remote -v
```

Question to answer: How many remotes do you have now?

Expected output

You should now see:

```
origin    https://github.com/YOUR-USERNAME/p390Practice.git (fetch)
origin    https://github.com/YOUR-USERNAME/p390Practice.git (push)
upstream  https://github.com/brittAnderson/p390Practice.git (fetch)
upstream  https://github.com/brittAnderson/p390Practice.git (push)
```

origin = your fork (you can push to this) **upstream** = original repo (you can only pull from this)

This naming convention (origin/upstream) is standard in the git community.

5.1.6.9 Fetch from Upstream

Task:

```
git fetch upstream
```

Question to answer: What does "fetch" do? Did it change any of your files?

Expected outcome

Fetch downloads commits from the remote but DOESN'T change your working files.

Think of it as: "Download new information but don't apply it yet."

After fetching, git knows what's new in the upstream repo, but your files remain unchanged until you merge.

5.1.6.10 View All Branches

Task:

```
git branch -a
```

Question to answer: What branches do you see?

Expected output

You might see:

- `* main` (your current local branch, marked with *)
- `remotes/origin/main` (the main branch on your fork)
- `remotes/upstream/main` (the main branch on the original repo)

Branches with `remotes/` are references to branches on GitHub, not local branches.

5.1.6.11 Pull from Upstream

Task:

```
git pull upstream main
```

Question to answer: What happened? Did you see "Already up to date" or were changes merged?

Expected outcome

`git pull` = `git fetch` + `git merge`

If the upstream repo has new commits, they're merged into your local branch. If there are no new commits, you see "Already up to date."

This is how you keep your fork synchronized with the original repository. You'll do this regularly to get new assignments, fixes, and updates.

5.1.6.12 Push Changes to Your Fork

Task:

```
git push origin main
```

Question to answer: What does this command do?

Expected outcome

This pushes your local commits to YOUR fork on GitHub (origin).

The flow is:

1. Pull from upstream (get professor's updates)
2. Work locally (make your changes)
3. Push to origin (your fork)

Check your fork on GitHub in a browser - you should now see your new commits there!

5.1.6.13 Verify on GitHub

Task:

1. Go to your fork: `https://github.com/YOUR-USERNAME/p390Practice`
2. Navigate to `my_practice/learning_notes.txt`
3. Click on the file to view it

Question to answer: Do you see your changes on GitHub?

Expected outcome

The file should contain both lines you added. The GitHub interface shows:

- The file contents
- When it was last modified
- The commit message
- Who made the change

If you don't see your changes, make sure you ran `git push origin main` successfully.

5.1.6.14 Understanding the Three-Repository Model

Task: Draw or describe the relationship between these three locations:

1. Your local repository (on your computer)
2. Your fork (origin, on GitHub)
3. The original repository (upstream, on GitHub)

Question to answer: Where can you make changes? Where can you only read?

The model

```
Upstream (brittAnderson/p390Practice)
    ↓ fork (one time)
    ↓ pull (regular updates)
    ↓
Origin (YOUR-USERNAME/p390Practice) on GitHub
     push/pull

Local (your computer)
```

You can:

- **Change freely**: Local repository
- **Push changes to**: Origin (your fork)
- **Pull from but not push to**: Upstream (original repo)

To contribute to upstream, you create a pull request.

5.1.7 Working in the gitnames Directory

5.1.7.1 Ensure You Have Latest Changes

Task

```
cd git_workspace/p390Practice
git status upstream main
git status origin main
```

Question to answer:

Why do we pull from upstream before starting work?

Expected outcome

This ensures that you have the latest changes from the original repo, including any files other students have added.

This avoids conflicts and ensures you're working with the most up-to-date version.

Always start your work session with this pull-merge-push sequence.

5.1.8 Navigate to gitnames Directory

Task:

```
ls
cd gitnames
ls
```

Question to answer: Do you see files from other students?

Expected outcome

You should see `.txt` files named after other students (if any have submitted their pull requests yet).

This directory is where everyone adds their file - it's a shared space for practicing collaboration.

5.1.9 Create Your File

Task: Create a file with your first name (replace YOURNAME):

```
touch YOURNAME.txt
echo "I am learning git and GitHub for Psychology 390" > YOURNAME.txt
echo "Git helps me track changes and collaborate effectively" >> YOURNAME.txt
cat YOURNAME.txt
```

Question to answer: Does the file exist? What does it contain?

Expected outcome

You created a personalized file in the gitnames directory.

Example: If your name is Alex, you'd create `alex.txt`.

This file will eventually be added to the original repository through a pull request - your first contribution to a collaborative project!

5.1.10 Check Status

Task:

```
git status
```

Question to answer: Where is your new file listed?

Expected output

Your file should appear under “Untracked files” in red.

Git sees the file but isn’t tracking it yet - you need to explicitly add it.

5.1.11 Stage Your File

Task:

```
git add gitnames/YOURNAME.txt
git status
```

Question to answer: What changed?

Expected outcome

The file moved from “Untracked files” to “Changes to be committed” (shown in green).

It’s now staged and ready to commit.

5.1.12 Commit Your Change

Task:

```
git commit -m "Add [YOURNAME] to gitnames directory"
```

Replace [YOURNAME] with your actual name.

Question to answer: Why is a good commit message important?

Expected outcome

Good commit messages: - Explain WHAT changed - Sometimes explain WHY (if not obvious) - Use present tense: “Add file” not “Added file” - Are specific: “Add alex.txt to gitnames” not “updated stuff”

When collaborating, clear commit messages help others understand the project’s history.

5.1.13 5.7 Check Log

Task:

```
git log --oneline -5
```

Question to answer: Can you see your commit?

Expected output

Your commit should be at the top of the list (most recent).

The `-5` flag shows only the last 5 commits for brevity.

5.1.14 Push to Your Fork

Task:

```
git push origin main
```

Question to answer: Where did this change go?

Expected outcome

Your commit is now on YOUR fork on GitHub (origin), but NOT yet on the original repository (upstream).

To get it into the upstream repo, you need to create a pull request (next exercise set).

Verify on GitHub: Go to `github.com/YOUR-USERNAME/p390Practice` and look in the gitnames directory.

5.1.15 Verify on GitHub

Task:

1. Open your fork in a browser: `https://github.com/YOUR-USERNAME/p390Practice`
2. Navigate to the gitnames directory
3. Find your file

Question to answer: Is your file visible on GitHub?

Expected outcome

You should see your file in the directory listing. If you click on it, you'll see its contents and the commit message.

If you don't see it, verify you successfully completed the push in 5.8.

5.1.16 Compare with Original

Task:

1. Open the original repo: `https://github.com/brittAnderson/p390Practice`
2. Navigate to gitnames
3. Look for your file

Question to answer: Is your file there?

Expected outcome

Your file is NOT in the original repository yet!

Your changes are:

- On your local computer
- On your fork (origin)
- NOT on the original repo (upstream)

The original repository is protected - you can't directly push to it. Instead, you request that the maintainer (me) pull your changes. That's called a "pull request."

5.2 Creating a Pull Request

5.2.1 Navigate to Your Fork

Task: Go to your fork on GitHub: `https://github.com/YOUR-USERNAME/p390Practice`

Question to answer: Do you see a yellow banner saying "This branch is X commits ahead of brittAnderson:main"?

Expected observation

If you see this banner, it means your fork has commits that aren't in the original repository.

This confirms you have changes ready to submit via pull request.

If you don't see this, make sure you pushed your changes in the previous exercise set.

5.2.2 Initiate Pull Request

Task:

1. Click "Contribute" (green button)
2. Click "Open pull request"
3. You'll be taken to the original repository with a comparison view

Question to answer: What two branches are being compared?

Expected view

You should see:

- **Base repository**: `brittAnderson/p390Practice` base: `main`

- **Head repository**: `YOUR-USERNAME/p390Practice` compare: `main`

This means: "I want to merge changes from MY main branch into the ORIGINAL main branch"

Below this, you'll see a list of commits and file changes - everything you're proposing to add.

5.2.3 Write Pull Request Description

Task:

1. Title: "Add [YOURNAME] to gitnames directory"
2. Description: Write a brief message explaining your change. For example:

This pull request adds my name file to the gitnames directory as requested in Exercise Set 5.

- Created YOURNAME.txt with learning reflections
- Following P390 course requirements

Question to answer: Why is a clear description helpful?

Best practices

Pull request descriptions should:

- Summarize what changed and why
- Reference any relevant issues or requirements
- Explain anything non-obvious
- Be polite and professional

The repository maintainer (me) will read this to understand your contribution before merging.

Think of pull requests as formal requests to add your work to a project.

5.2.4 Create the Pull Request

Task:

1. Review your changes in the "Files changed" tab
2. Click "Create pull request"
3. Note the pull request number (e.g., #42)

Question to answer: What number is your pull request?

Expected outcome

You've created your first pull request!

The PR now appears in the original repository's pull request list. The maintainer (me) can:

- Review your changes
- Leave comments
- Request modifications
- Merge (accept) your changes
- Close without merging (reject)

Your changes are still only in your fork until the PR is merged.

5.2.5 View Your Pull Request

Task:

1. Go to `https://github.com/brittAnderson/p390Practice/pulls`
2. Find your pull request in the list
3. Click on it to see details

Question to answer: What information is displayed on the pull request page?

Expected view

You should see:

- Title and description
- Status (Open/Merged/Closed)
- Commits included
- Files changed
- Discussion/comments
- Checks (automated tests, if configured)

This is the collaboration interface where you and the maintainer can discuss the changes.

5.2.6 Understanding Pull Request Status

Task:

Check your pull request status. It will be one of:

- **Open** (pending review)
- **Merged** (accepted and integrated)
- **Closed** (not accepted)

Question to answer: What's the current status of your PR?

What happens next

The repository maintainer will: 1. Review your changes 2. May leave comments or request changes 3. Will eventually merge (accept) or close (reject)

When merged, your changes become part of the original repository!

You'll get email notifications about any activity on your PR.

5.2.7 After PR is Merged

Task:

Once your PR is merged (you'll get a notification), update your local repository:

```
cd ~/git_workspace/p390Practice
git pull upstream main
git push origin main
```

Question to answer: What does this accomplish?

Expected outcome

This synchronizes everything:

1. `git pull upstream main` - downloads the merged changes from the original repo (which now includes your contribution AND contributions from other students)
2. `git push origin main` - updates your fork to match

Now all three locations (local, origin, upstream) have the same code.

5.2.8 See Everyone's Contributions

Task:

```
cd ~/git_workspace/p390Practice/gitnames
ls
```

Question to answer: How many files are there now?

Expected outcome

You should see files from multiple students - everyone who has had their pull request merged.

This is collaborative version control in action! Everyone contributed their file, and git managed merging all changes together.

If there were any conflicts (two people editing the same line), the maintainer would have resolved them during the merge.

5.2.9 View Merged Commit

Task:

```
git log --oneline --all --graph
```

Question to answer: Can you identify the merge commit that brought your changes into the original repository?

Expected output

You should see a graph showing:

- Your commits
- Other students' commits
- Merge commits (where pull requests were merged)

The graph visualizes the project's collaborative history.

Press `q` to exit.

5.3 Opening an Issue

5.3.1 Understanding Issues

Task:

Go to `https://github.com/brittAnderson/uwlooP390/issues`

Question to answer: What kinds of things do you see reported in the issues?

Expected observations

Issues are used for:

- Bug reports ("The install instructions don't work on Mac")
- Feature requests ("Could we add examples for SPSS users?")
- Questions ("How do I set up RStudio?")
- Documentation problems ("The README is unclear about X")
- Discussions about the project

Issues are NOT for:

- General chat
- Personal technical support unrelated to the project
- Spam

Think of issues as a project's to-do list and discussion forum combined.

5.3.2 Navigate to Practice Repository Issues

Task:

Go to `https://github.com/brittAnderson/p390Practice/issues`

Question to answer: Are there any open issues? What are they about?

Expected view

You might see:

- Issues from other students
- My responses to issues
- Closed issues (resolved problems)

This is where the class can ask questions and report problems.

5.3.3 Create an Issue

Task:

1. Click "New issue"
2. Title: "Practice issue from [YOUR NAME]"
3. Description: Write a brief message. For example:

This is a practice issue to demonstrate understanding of GitHub's issue system.

I have successfully:

- Created a GitHub account
- Forked the practice repository
- Created a pull request
- Opened this issue

This issue can be closed after verification.

4. Click "Submit new issue"

Question to answer: What number is your issue?

Expected outcome

Your issue now appears in the issue list. Note the issue number (e.g., #15).

Issues are numbered sequentially across the repository. Your pull request and issues share the same numbering system.

5.3.4 Record Your Issue Number

Task:

In a text file on your computer, record:

- Your GitHub username
- Your pull request number
- Your issue number

Save this as `github_completion.txt`

Question to answer: Why might this information be important?

Expected outcome

In a course setting, this information helps the instructor:

- Match GitHub activity to students
- Verify that you completed the exercises
- Track contributions

In real projects, you'd reference issue numbers in commits and pull requests to link related work.

5.3.5 Use Issues Constructively

Task:

Read through some issues on the course repository: `https://github.com/brittAnderson/uwlooP390/issues`

Question to answer: What makes a good issue report?

Best practices

Good issues:

- Have clear, descriptive titles
- Explain the problem or request completely
- Include steps to reproduce (for bugs)
- Are respectful and professional
- Stay on topic

Bad issues:

- "It doesn't work" (too vague)
- Off-topic or spam
- Rude or demanding

- Duplicate existing issues

Before creating an issue, search to see if someone else already reported the same problem.

5.3.6 Comment on an Issue

Task:

Find any open issue (yours or someone else's) and add a comment:

- If it's your issue: Add "Issue number recorded for course requirements"
- If it's someone else's: Add a helpful observation or supportive comment

Question to answer: How does commenting help collaboration?

Expected outcome

Comments on issues allow:

- Discussion of solutions
- Requests for clarification
- Sharing of workarounds
- Building on others' ideas
- Keeping everyone informed

Issues become threaded conversations that document how problems were solved.

5.3.7 Understand Issue Labels

Task:

Look at issues on the main course repo and note any labels (like "bug", "question", "enhancement")

Question to answer: How do labels help organize issues?

Expected observation

Labels categorize issues:

- **bug**: Something isn't working
- **enhancement**: Feature request
- **question**: Need help or clarification
- **documentation**: Docs need improvement
- **good first issue**: Beginner-friendly

Labels help people find relevant issues and prioritize work.

Repository maintainers assign labels, though some repos let contributors suggest them.

5.3.8 Close Your Practice Issue

Task:

1. Go to your practice issue
2. Add a comment: "Exercise completed, closing issue"
3. Click "Close issue"

Question to answer: What happens to closed issues?

Expected outcome

Closed issues:

- Are marked as resolved
- Remain visible in the issue list (with "Closed" filter)
- Can be reopened if needed
- Are searchable forever

Closing doesn't delete - it just marks as done. The conversation history is preserved for future reference.

5.3.9 Using Issues for Learning

Task:

Think about situations where you might open a real issue on the uwlooP390 repository.

Question to answer:

What kinds of problems or questions would be appropriate to report as issues?

Appropriate uses

Good reasons to open an issue:

- Assignment instructions are unclear
- You found a typo in course materials
- Installation instructions don't work
- You have a suggestion for improvement
- You need clarification on requirements

The issue tracker is a resource for all students - if you're confused, others probably are too!

5.3.10 Reflection on Collaborative Features

Task:

Think about how pull requests and issues work together.

Question to answer:

How do pull requests and issues relate? How might you reference an issue in a pull request?

The connection

Pull requests and issues work together:

- Issues identify problems
- Pull requests propose solutions

You can reference issues in PR descriptions:

- "Fixes #23" - automatically closes issue #23 when PR is merged
- "Relates to #15" - links to issue without closing it

This creates a traceable history: from problem identification (issue) to solution (PR) to implementation (merge).

5.4 Real-World Workflow Practice

5.4.1 8.1 Create a Work Branch (Optional Advanced)

Task:

```
cd ~/git_workspace/p390Practice
git checkout -b my-experiments
```

Question to answer: What did this do?

Expected outcome

You created and switched to a new branch called "my-experiments".

Branches let you work on different things independently: - `main` branch: stable, synchronized code - `my-experiments` branch: where you try new things

This is optional but common in professional workflows. Changes on this branch don't affect main until you explicitly merge them.

5.4.2 8.2 Practice Making Changes

Task: Create a practice file:

```
mkdir experiments
echo "Experiment 1: Testing git workflow" > experiments/notes.txt
git add experiments/notes.txt
git commit -m "Add experiment notes"
```

Question to answer: Is this change on your main branch?

Expected outcome

No! This commit is only on the `my-experiments` branch.

Type `git checkout main` to switch back to main, then `ls` - the experiments directory isn't there.

Type `git checkout my-experiments` and `ls` again - now it appears.

Branches are independent lines of development.

5.4.3 8.3 Merge Branch into Main (Optional Advanced)

Task:

```
git checkout main
git merge my-experiments
ls
```

Question to answer: Is the experiments directory now in main?

Expected outcome

Yes! Merging brought the changes from `my-experiments` into `main`.

This is how you incorporate experimental work into the main codebase once you're satisfied with it.

Delete the branch if done: `git branch -d my-experiments`

5.4.4 8.4 Simulate Regular Workflow

Task: Practice this sequence (which you'll do often):

```
# Start of work session
git pull upstream main  # Get latest from course repo
git push origin main    # Update your fork

# Do your work
echo "New learning point" >> my_practice/learning_notes.txt
git add my_practice/learning_notes.txt
git commit -m "Add new learning point"
```

```
# End of work session
git push origin main    # Save your work to GitHub
```

Question to answer: Why is this sequence important?

Expected outcome

This workflow ensures:

1. You always have the latest changes
2. Your work is backed up on GitHub
3. You can collaborate without conflicts

Make this a habit:

- Start each session: pull upstream, push to origin
- Work locally: add, commit frequently
- End each session: push to origin

5.4.5 8.5 Understanding Merge Conflicts

Task: Read about merge conflicts (you don't need to create one now):

A merge conflict happens when:

- Two people edit the same line of the same file
- Git can't automatically determine which version to keep

Question to answer: How would you avoid conflicts in collaborative work?

Prevention strategies

To minimize conflicts:

- Pull from upstream frequently
- Communicate with collaborators about what you're working on
- Commit and push regularly
- Work on different files when possible
- If you must edit the same file, work on different sections

When conflicts do occur, git marks them in the file and you manually choose which version to keep.

5.4.6 8.6 Check Remote Relationship

Task:

```
git remote -v
git branch -vv
```

Question to answer: What does `-vv` show you?

Expected output

`git branch -vv` shows:

- Which branch you're on
- The last commit
- Which remote branch it tracks

Example:

```
* main abc1234 [origin/main] Latest commit message
```

This confirms your local main tracks origin/main (your fork's main branch).

5.4.7 8.7 Practice git status Awareness

Task: Make it a habit to run `git status` frequently while working.

Question to answer: What are the possible states you might see?

Common states

- **Clean working tree**: No changes since last commit
- **Untracked files**: New files git isn't tracking (red)
- **Changes not staged**: Modified tracked files, not added (red)
- **Changes to be committed**: Staged files, ready to commit (green)
- **Branch ahead/behind**: Local differs from remote

`git status` is your most important diagnostic command. Use it constantly!

5.4.8 8.8 View What Changed

Task:

```
echo "Another line" >> my_practice/learning_notes.txt
git diff my_practice/learning_notes.txt
```

Question to answer: What does `git diff` show?

Expected output

`git diff` shows exactly what changed:

- Lines removed (preceded by `-`, shown in red)
- Lines added (preceded by `+`, shown in green)
- Context lines (unchanged)

This lets you review changes before committing.

Use:

- `git diff` - unstaged changes
- `git diff --staged` - staged changes
- `git diff HEAD` - all changes since last commit

5.4.9 8.9 Undo Mistakes Safely

Task: Learn these safety commands (don't run them unless needed):

Discard unstaged changes to a file:

```
git checkout -- filename
```

Unstage a file (keep the changes):

```
git reset HEAD filename
```

Amend the last commit (before pushing):

```
git commit --amend -m "New message"
```

Question to answer: Why is it important to know how to undo changes?

Safety knowledge

Everyone makes mistakes:

- You edit the wrong file
- You commit with a bad message
- You stage files you didn't mean to

Knowing how to safely undo changes prevents panic and destructive solutions (like deleting the repository and starting over).

IMPORTANT: These undo commands are safe BEFORE pushing. After pushing, you need different approaches.

5.4.10 8.10 Final Workflow Check

Task: Verify your complete understanding by describing each step:

1. How do you get changes from the original course repo?
2. How do you save your work locally?

3. How do you back up your work to GitHub?
4. How do you propose changes to the original repo?
5. How do you report problems or ask questions?

Complete workflow summary

1. **Get updates**: `git pull upstream main`
2. **Save locally**: `git add files`, then `git commit -m "message"`
3. **Back up**: `git push origin main`
4. **Propose changes**: Create a pull request on GitHub
5. **Report problems**: Open an issue on GitHub

Additional commands you've learned:

- `git status` - check what's changed
- `git log` - view history
- `git diff` - see specific changes
- `git remote -v` - check remotes
- `git branch` - manage branches

This is professional version control!

5.5 Self-Assessment Checklist

After completing these exercises, you should be able to:

Basic Operations:

☐ Create and configure a GitHub account

☐ Fork repositories

☐ Clone repositories to your computer

☐ Check repository status (`git status`)

☐ View commit history (`git log`)

Making Changes:

☐ Create and edit files

☐ Stage changes (`git add`)

☐ Commit changes with good messages (`git commit`)

☐ Push changes to your fork (`git push`)

Collaboration:

- ☐ Add upstream remote
- ☐ Pull from upstream (`git pull upstream main`)
- ☐ Keep fork synchronized
- ☐ Create pull requests
- ☐ Open and comment on issues

Understanding:

- ☐ Explain the difference between local, origin, and upstream
- ☐ Describe the purpose of committing, pushing, and pulling
- ☐ Understand why pull requests are necessary
- ☐ Know when to use issues vs pull requests

5.6 Common Pitfalls and Solutions

5.6.1 "Permission denied" when pushing

Problem: Trying to push to upstream instead of origin

Solution: Always `git push origin main`, never push to upstream

5.6.2 "Please commit your changes or stash them"

Problem: Trying to pull/checkout with uncommitted changes

Solution: Either commit your changes first, or use `git stash` to temporarily save them

5.6.3 "Your branch is behind origin/main"

Problem: Someone else pushed to your fork (or you pushed from another computer)

Solution: `git pull origin main` to get those changes

5.6.4 Can't see your pushed changes on GitHub

Problem: You committed but didn't push

Solution: Run `git push origin main`

5.6.5 Accidentally edited files in the original repo

Problem: Cloned upstream instead of your fork

Solution: Clone from YOUR fork, then add upstream as a second remote

5.6.6 "Merge conflict"

Problem: Same file edited in two places

Solution: Edit the file to resolve conflicts (git marks them with <<<<<<<), then add and commit

5.7 Git Commands Quick Reference

5.7.1 Setup

```
git config --global user.name "Your Name"
git config --global user.email "email@example.com"
```

5.7.2 Daily Workflow

```
git status              # Check what's changed
git pull upstream main  # Get latest from course repo
git add filename        # Stage a file
git commit -m "msg"     # Save changes locally
git push origin main    # Upload to your fork
git log --oneline       # View history
```

5.7.3 Repository Management

```
git clone url           # Download a repository
git remote -v           # List remotes
git remote add name url # Add a remote
git branch              # List branches
git diff                # See unstaged changes
```

5.7.4 Collaboration

- **Pull Request**: On GitHub, after pushing to your fork
- **Issues**: On GitHub, click "Issues" → "New issue"

5.8 Getting Help

5.8.1 In the Terminal

```
git help <command>      # Full manual
git <command> --help    # Same as above
git <command> -h        # Brief help
```

5.8.2 Online Resources

- GitHub Documentation
- Git Documentation
- GitHub Skills - Interactive tutorials
- Course repository issues: `https://github.com/brittAnderson/uwlooP390/issues`

5.8.3 Best Practices

- Commit often (every logical unit of work)
- Write clear commit messages
- Pull before starting work
- Push at the end of every session
- Use issues to ask for help

5.9 Troubleshooting

5.9.1 Need to Start Over?

```
# Delete local repository
cd ~/git_workspace
rm -rf p390Practice

# Clone again
git clone https://github.com/YOUR-USERNAME/p390Practice.git
cd p390Practice
git remote add upstream https://github.com/brittAnderson/p390Practice.git
```

5.9.2 Completely Lost?

1. Save any files you care about outside the repository
2. Delete the repository folder
3. Start fresh with cloning

4. Ask for help via issues on the course repo

Remember: Git has a learning curve, but it's the industry standard for a reason. Every professional programmer uses it. These skills are directly transferable to:

- Research collaborations
- Graduate work
- Industry positions
- Open source contributions
- Personal projects

You're building valuable, marketable skills!

Chapter 6

Making a Website with Quarto and Github Pages

Having a professional website is increasingly important for researchers and scientists. It serves as a central location for your publications, projects, and professional information. When colleagues, potential collaborators, or employers search for you online, your website provides a controlled first impression. This tutorial will guide you through creating and hosting your own website using Quarto and GitHub Pages.

6.0.1 Hosting

To have a website viewable by the world it must be hosted on a server that is viewable by the world. For example, it cannot be behind an institutional firewall (as are most of the computers at the University of Waterloo). And if you host on a University or Company server you have the potential of being limited in what you can share, how often you can update, or you may be required to adhere to certain styles and formats mandated by the institution's brand. One way to avoid this is to find an external hosting site. A modest fee would allow you to have a virtual server in the cloud, but the example we will use here to get you started is a free service available to those with Github accounts: Github Pages.

6.0.2 Testing Github Pages

Assuming you have an account you should be able to create a minimal webpage in a few steps.

1. Go to personal dashboard and create a new repo as instructed on the Github Pages website. The repo must be named your username.github.io.
2. Use your terminal to clone this repo to your local computer.

3. cd into the correct directory using your terminal (inside VS Code is fine). And using the *terminal* do `echo "hello world" > index.html.`
4. Do the usual add, commit, and push and verify your web site works at <username.github.io>.

You now have a website! To view it go to yourusername.github.io

6.0.3 Converting to Quarto

While a basic HTML page demonstrates that GitHub Pages is working, Quarto provides a more powerful framework for creating professional websites. Quarto allows you to write content in Markdown, automatically generates navigation, and provides professional themes. The following steps will convert your basic site to a Quarto website.

6.0.4 Quarto Setup

1. **Remove the test HTML file**:

   ```
   rm index.html
   ```

2. **Initialize a Quarto website project** in your repository folder (using the command in your terminal):

   ```
   quarto create project website .
   ```

   Choose "website" when prompted, and confirm overwriting the directory.

3. **Preview your site locally** to verify it's working:

   ```
   quarto preview
   ```

   Your browser should open displaying the default Quarto website.

4. **Customize the homepage** by editing `index.qmd` in VS Code. Replace the default content with your information:

   ```
   ---
   title: "[Your Name]"
   ---

   ## About

   [Your academic position and affiliation]

   ## Research Interests

   [Your research areas and current projects]

   ## Contact
   ```

```
[Your professional contact information]
```

5. **Configure for GitHub Pages deployment** by editing `_quarto.yml`:

```
project:
  type: website
  output-dir: docs
```

6. **Build and deploy your site**:

```
quarto render
git add .
git commit -m "Initial Quarto website"
git push
```

7. **Configure GitHub Pages**: Navigate to your repository Settings → Pages → Source: Deploy from branch → Branch: main → Folder: /docs → Save

Your website should be available at .github.io within a few minutes.

6.1 Customization Exercise

Practice modifying your website by completing these tasks:

- Create an About page by adding a new file called `about.qmd`
- Change the website theme by modifying the `theme` field in `_quarto.yml` (available themes include: cosmo, darkly, flatly, journal, litera, lumen, lux, materia, minty, morph, pulse, quartz, sandstone, simplex, sketchy, slate, solar, spacelab, superhero, united, vapor, yeti, zephyr)
- Add a navigation menu by updating the `navbar` section in `_quarto.yml`
- Include an image or figure on your homepage

Use `quarto preview` to test changes locally before committing and pushing to GitHub.

Chapter 7

Coding General

> For better or worse, nearly every neuroscientist is now an amateur software engineer to some degree. Source

Simply put, this is you telling the machine what to do. It is usually done via the writing of a file in plain text that is then processed in various ways to yield machine code that the computer actually uses. It is possible to program in machine code, but it is not very user friendly. If this sounds familiar, it is because writing qmd or LaTeX is a form of coding. You have already been doing it.

7.1 Languages

There is more than one language available for programming your computer and each has its pros and cons. Some are "low level" or close to the metal (see for example, rust or zig), while others, the ones we will use in this course are considered high level, and abstract away many of the details (such as how much memory to allocate or how to clean-up (garbage collection) unused data from memory).

When evaluating the fitness of a new programming language ask yourself not only whether it meets your needs today, but does it look likely to be available tomorrow, or five years from tomorrow? It is also, in general, better to choose a more general purpose language that you can use for more than one particular task. This will allow you to leverage your knowledge of how to get things done in one domain for how to get things done in another domain. And it is likely that you will learn general patterns that may help you pick up additional programming languages more quickly if, in the future, you need to change.

7.2 Common Programming Constructs

Some of the basic vocabulary may help you as you begin to program. The succeeding chapters will give you some specifics, experience, and practice with both Python and R.

7.2.1 Variables

Variables are things (lists, values, names, and such) that you want to give a name to. You might do this because you think you are going to want to do something with them later and it might be more convenient to have a short, memorable name for referring to them.

7.2.2 Functions

Functions have inputs and output. The inputs may be called parameters or arguments. The outputs may be called return values. What the function does is describe the processing of the inputs to the outputs. Think of it as a factory. It takes in raw materials (inputs) and produces an output. You are describing a factory when you write a function.

7.2.3 Interpreted

An interpreted language is one where you type in code one line at a time and a program converts that to machine code, evaluates it, and prints the return value for you to read. This is an interactive mode. Python is an example of an interpreted language.

7.2.4 Compiled

You write the whole program. Another program reads your code and all at once, and possibly requiring several passes and linking to other code elsewhere produces a single compiled version for the machine to read and use. C is an example of a compiled language. To some extent so is LaTeX.

7.2.5 Packages/Libraries

Many programming languages are very basic in what they offer. In order to do the cool stuff you will need additional collections of code, some of it may even be written in a different programming language. The colletions are often called packages or libraries that you import, source, or load. `ggplot2` is an example of an R plotting library. `matplotlib` is similar for python.

7.2.6 Types

Many programming languages have a notion of a datatype. `1` can be a number, but "1" can also be a character that you are typing. It doesn't make sense to

add one, the number, to one, the character, but it might make sense to `1 + 1` for either. You would expect 2 for numbers and maybe "11" for characters. Some programming languages, such as python, type *dynamically*. They figure out what type to treat a value as they read your code. They try to deduce it from the context. Other languages, such as Haskell, are *statically* typed. They require that everything have a type and that it never changes. Dynamically typed languages are often easier to get started with, but statically typed languges can prove helpful when writing larger pieces of software.

Other sorts of data structures that you might run across in this course are: - Lists,

- Tuples,
- Data.Frames

7.2.7 Loops

Loops are a common programming construct. They allow you to repeat an action multiple times. For example, if you wanted to print a number you could repeatedly use something like `print(my_number)` as long as you changed my_number in between each use.

Common varieties of loops are the `for` and `while` loop.

7.2.8 Virtual Environments

You will often have software installed on your computer that you use for multiple purposes. There is also software on your computer that is used by your operating system. If you download different versions of that software for your own use you may find that your programs do not run correctly, or even worse that your system does not work properly.

To avoid such problems you may want to "sandbox" your development environment. Conflicts are less common for R, but very common for python. Such sandboxes are often called `venv`.

A virtual environment lets you install software that only works locally, and cannot interfere with your global system software or other virtual environments you may have constructed for other projects.

7.2.9 Assignment and Equality Are Different in Programming

Assignment means you are assigning a name to something. Equality means that you are testing if two things are the same. One equals sign does not equal two equals sign. That is `= != ==`. Exclamation points are often used to *negate* something (turn true to false and vice versa).

```
a = 2
print(a == 3)
```

```
False
```

In the first line we *assign* the value of two to the variable `a`. In the next line we test if a and 3 are in fact equal to each other. A "boolean" value is returned: false.

Chapter 8

R

8.1 Installation

See the operating system specific appendices.

8.1.1 Test your installation

```
R cmd --version
```

```
R version 4.5.2 (2025-10-31) -- "[Not] Part in a Rumble"
Copyright (C) 2025 The R Foundation for Statistical Computing
Platform: x86_64-pc-linux-gnu

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under the terms of the
GNU General Public License versions 2 or 3.
For more information about these matters see
https://www.gnu.org/licenses/.
```

You should see some information about you R version.

8.2 Interfacing with VS Code

Restart VS Code and install the R extension (look to the left panel for the extension launcher; it looks like a little set of squares).

8.2.1 Resources

R for Data Science is a text book on data science that uses R. The author is a prolific R package author and the itself written in Quarto. It includes many examples that you can test by cutting and pasting into your IDE.

8.3 Installing R Packages

There are many tools for this. Some of them are GUI tools, and they will at first appear easier. But if your library starts to grow, or you need to install particular versions of packages, old packages, or packages from Github and not on CRAN, then you will be much happier in a R interpreter.

To create an R interpreter in VS Code you can simply launch a new terminal and then type R and . You should see an R welcome message. Code like the snippet to follow shows a basic installation exchange.

```
install.packages("tidyverse",repos='https://mirror.csclub.uwaterloo.ca/CRAN/')
library{tidyverse}
```

Figure 8.1

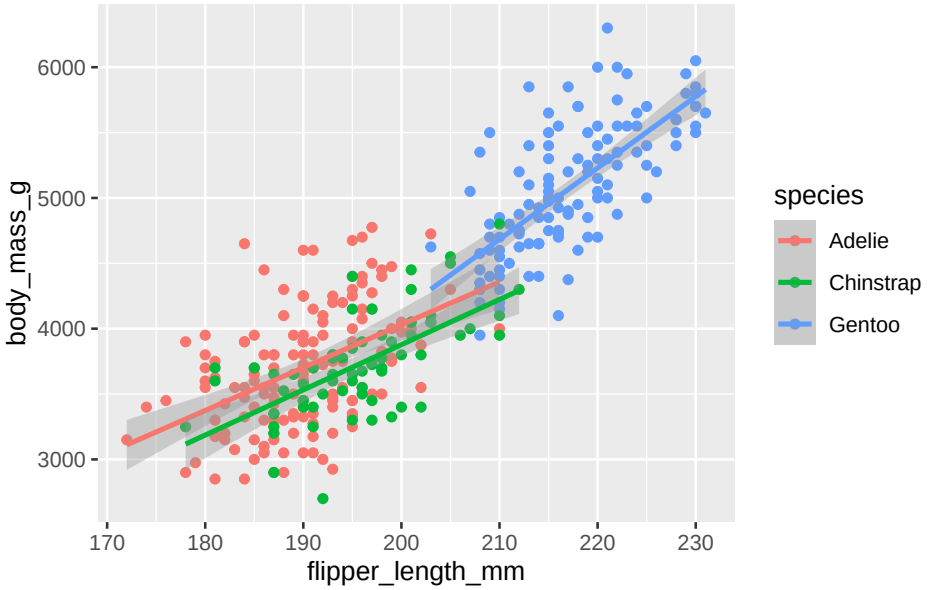
The tidyverse is a very popular collection of R code that itself will depend on many additional pieces of R code and other packages, some R and some not.

8.4 Mix R Code with Text for Reproducible and Documented Workflows

Here is an example of embedding R code in a qmd document. In this code we download and install a package if necessary (the `if` statement). Then we use that package to load some data and generate a plot. The plot you see in this document is the result of that code. I did not cut and paste a png.

```
if(!require(tidyverse)){
    install.packages("tidyverse",repos='https://mirror.csclub.uwaterloo.ca/CRAN/')
}
if(!require(palmerpenguins)){
    install.packages("palmerpenguins",repos='https://mirror.csclub.uwaterloo.ca/CRAN/')
}
library(tidyverse)
library(palmerpenguins)

ggplot(
  data = penguins,
  mapping = aes(x = flipper_length_mm, y = body_mass_g, color = species)
) +
  geom_point() +
  geom_smooth(method = "lm")
```



8.5 R Types

As mentioned in the code chapter, programming language values are typically typed. R in particular has many complicated data types that represent the output of functions and statistical analyses. If things are getting confusing you might want to check on the type of your data.

```
a = 1
typeof(a)
[1] "double"
```

Figure 8.2: Checking the type of a number in R.

```
tpl = c(1,2)
lst = list("firstName" = 'Britt', "lastName" = 'Anderson')
df = data.frame('fn' = c("bob","jane","griffin"),"gndr" = c('m','f','o'))
df
       fn gndr
1     bob    m
2    jane    f
3 griffin    o
```

Figure 8.3: Each of three datatypes in R are demonstrated.

8.6 Data.Frames

`Data.Frames` are particularly central to the R analysis model and are a killer feature. Think of `data.frames` as a supercharged spreadsheet with columnn names.

```
is.data.frame(cars)
```

```
[1] TRUE
```

```
head(cars)
```

```
  speed dist
1     4    2
2     4   10
3     7    4
4     7   22
5     8   16
6     9   10
```

`cars` is a data set built in the R language. We use the `is.data.frame` command to ask if a variable is or is not a data.frame.

8.7 Selecting Data

A bit of code that tests for whether something is true or false can be called a *predicate*. Predicates are important when having your code do something conditionally. If this is true then do this, otherwise do that. This common construct of "if-then-else" exists in most programming languages. In the next code snippet we use the conditional expressions to select on a subset of our data. You might use something similar to figure out how many of the participants in an experiment were Arts students and under 20 years of age.

```
length(cars$dist[cars$speed > 10 & cars$speed < 20])
```

```
[1] 29
```

Figure 8.4: How many cars are there that can go faster than 10, but not more than 20?

What is going on here? 1. `cars` is the name of the data frame. 2. We access a *column* of the data frame with the dollar sign notation `cars$dist`. You can see the names of the columns in a data frame with `names(cars)`. 3. We use the square brackets to *index* the column of data we are interested in. Here we do not use specific numbers, but rather we use a *boolean* to compare the values in a column and we only include the *TRUE* values. Here we ask for all the indices where the speed is greater than 10, but less than 20, and we use those indices to get the values for the `dist` column.

8.8 Practice

While qmd files and Quarto are great for writing documents with code in it, they are not idea for exploring in a quick and interactive way. For that use a terminal or console. In VSCode you have a menu that will create a new terminal where you can start R with the command `R`.

1. Sort (or `order`) cars by the `dist` variable.
2. Find the mean and standard deviation of the speed of the cars.
3. Are there other datasets?

   ```{r}
   library(help="datasets")

   ```

4. Open any of the datasets that catches your eye.
5. What are the column names?
6. How many rows?
7. What is the *comment* designator for R?
8. What is the ending extension of an R script?

8.9 Looping in R

> **Tip**
>
> Loops are more common in python than R. R often allows you to "map" over data, and you will not need to loop, but when you do …

`For` loops are the most common looping construct.

```
ml <- seq(1:10)
for  (m in ml) {
  print(ml)
}
```

```
 [1]  1  2  3  4  5  6  7  8  9 10
 [1]  1  2  3  4  5  6  7  8  9 10
 [1]  1  2  3  4  5  6  7  8  9 10
 [1]  1  2  3  4  5  6  7  8  9 10
 [1]  1  2  3  4  5  6  7  8  9 10
 [1]  1  2  3  4  5  6  7  8  9 10
 [1]  1  2  3  4  5  6  7  8  9 10
 [1]  1  2  3  4  5  6  7  8  9 10
 [1]  1  2  3  4  5  6  7  8  9 10
```

```
 [1]  1  2  3  4  5  6  7  8  9 10
 [1]  1  2  3  4  5  6  7  8  9 10
```

8.10 More Practice

1. Edit the above so that it prints the individual number each time it goes through the loop, and not the whole list.
2. Create a list of at least 8 individual characters. 1.Make sure they are **not** in alphabetical order 2.Print the letters one at a time. 3.Print the letters sorted alphabetically one at a time, but *do not* overwrite your original list. 4.Print the letters from both lists with a `format` command that says which position the letter is in.

```
myName = "brittAnderson"
myList = unlist(strsplit(myName,""))

for (l in myList){
  print(l)
  }

for (l in myList[order(myList)]){
  print(l)
  }

i = 1
for (n in order(myList)){
  t  <- sprintf("The %.0fth letter of myList is: %s, but is %s in the sorted list.",i,n
  print(t)
  i = i+1
  }
```

8.11 Writing a Function in R

```
myadd  <- function(x,y) {                                    ①
  return(x+y)                                                ②
  }
```

① The arrow is used to assign something to a name. Other languages usually use the equals sign. The keyword `function` tells R you are creating a function, and the parentheses following establishes the number of inputs and the nicknames you will use in your function.

② `return` specifies what we want to get back from the function.

Now we can use the function we defined with particular inputs.

```
myadd(2,3)
```

```
[1] 5
```

Chapter 9

Python

9.1 Installation

See OS specific appendices.

9.2 Packages

Unlike R, you do not install python packages from within an interpreted environment. Worse, the python package system has different tools. Recently, `pip` has been the most commonly used, and there is an interface to it through `uv`. Conflicting packages are a real risk. Use a virtual environment. Virtual environments are also available via the `uv` interface.

9.2.1 Using uv to create an env and install local packages

First, initiate the virtual environment and declare the python version to use. You will first create the environment. Then you will **activate** it. I find this far easier in a shell/terminal/command-line environment.

```
uv venv --python 3.10
source .venv/bin/activate
```

NB: As of 2025 the most recent version of python that works well with psychopy is said to be 3.10.

This will create a new *hidden* (directories starting with a "dot" like `.venv` will not display on many operating systems unless you specifically ask to see them.) directory called `.venv` inside the directory where you invoked that command. To activate the virtual environment you do `source <path-to>/.venv/bin/activate`.

When active your subsequent python command should refer to this python installation.

To install packages the code would look like (again in the shell):

```
uv pip install numpy matplotlib jupyter pandas
```

I picked these packages, because I know they will come in handy later.

9.2.2 Testing Your Python Package Installation

> **Warning**
>
> You may need to install the "reticulate" package for R if you want to run both python and R code in the same document as I am trying to do here.

And you will.

> **Warning**
>
> Use your .venv. If you are using VSCode you will have to tell VSCode to use your python environment. The shortcut Ctrl-Shift-p will open up the command menu and you can search for python: select interpreter. Guide it to the python version in your *activated* venv.

If you did all the above then this should generate a rather cool looking set of figures.

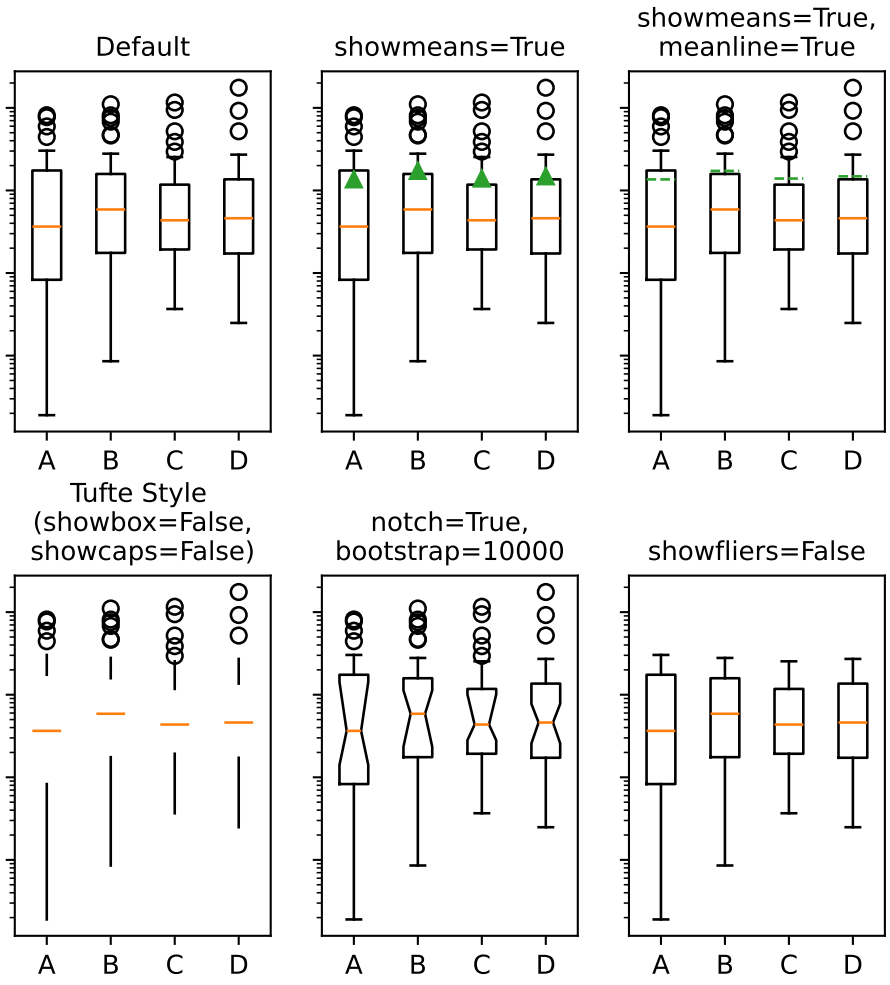
```
import matplotlib.pyplot as plt
import numpy as np

# fake data
np.random.seed(19680801)
data = np.random.lognormal(size=(37, 4), mean=1.5, sigma=1.75)
labels = list('ABCD')
fs = 10  # fontsize
fig, axs = plt.subplots(nrows=2, ncols=3, figsize=(6, 6), sharey=True)
axs[0, 0].boxplot(data, tick_labels=labels);
axs[0, 0].set_title('Default', fontsize=fs);
axs[0, 1].boxplot(data, tick_labels=labels, showmeans=True);
axs[0, 1].set_title('showmeans=True', fontsize=fs);
axs[0, 2].boxplot(data, tick_labels=labels, showmeans=True, meanline=True);
axs[0, 2].set_title('showmeans=True,\nmeanline=True', fontsize=fs);
axs[1, 0].boxplot(data, tick_labels=labels, showbox=False, showcaps=False);
tufte_title = 'Tufte Style \n(showbox=False,\nshowcaps=False)';
axs[1, 0].set_title(tufte_title, fontsize=fs);
axs[1, 1].boxplot(data, tick_labels=labels, notch=True, bootstrap=10000);
axs[1, 1].set_title('notch=True,\nbootstrap=10000', fontsize=fs);
axs[1, 2].boxplot(data, tick_labels=labels, showfliers=False);
```

```
axs[1, 2].set_title('showfliers=False', fontsize=fs);

for ax in axs.flat:
    ax.set_yscale('log');
    ax.set_yticklabels([]);

fig.subplots_adjust(hspace=0.4)
```



Warning

Python cares about whitespace. It is strongly recommended that you only use spaces and not tabs when writing python code.

9.3 Loops in Python

This is a snippet of code to demo the use of a for loop in python. Note how we can use the `enumerate` function to return two loop variables (`i` and `j`) that we set at one time.

```
my_name = "britt"                                                    ①
my_name_as_list = list(my_name)                                      ②
for i, j in enumerate(my_name_as_list):                              ③
  print((i, j))
```

```
(0, 'b')
(1, 'r')
(2, 'i')
(3, 't')
(4, 't')
```

Some things to note: 1. Assignment in python does not use the arrow like R does. 2. Strings and lists are different *types* of data. I have to coerce my string to a list.

Figure 9.1: What a for loop might look like in python.

9.3.1 Loops in Python Make Use of Iterables

Python refers to things called "iterables." To iterate is another way of saying that you can keep doing the same thing over and over. Imagine a bowl of ice cream. It is "eatable". You take one spoonful, and then another, and keep taking spoonfuls until the bowl is empty.

Indexing is how you get the location of an element in a list. Think of the row or column number in a spread sheet program, but indexes can be more powerful, and can be nested easily. In many programming languages, but sadly not all, **indexes start at 0**. Different programming languages will have slightly different syntax.

```
name_dict = {'firstName' : 'Britt', 'lastName' : 'Anderson'}     ②
my_list = list(range(1, 10))                                     ①
print(name_dict['firstName'])
print(my_list)
print(my_list[0])
print(my_list[-1])
print(my_list[0:4])
```

9.3.2 Additional Examples of Programming Constructs in Python

Another `for` loop in python

① Notice that here we use `range` where we use `seq` in R. There is usually a way to do something similar in both languages, but they may not have the same name.

② Also note that I am using a python dictionary. It has keys and values. It can be a nice way to handle data files.

```
Britt
[1, 2, 3, 4, 5, 6, 7, 8, 9]
1
9
[1, 2, 3, 4]
```

Figure 9.2: The use of the `-1`as an index is a python trick for getting the last element in a list. Think of the list as a circle. If you count backwards from a list you will get to the beginning eventually (index 0) if you went back one more step (-1) you would circle back to the end of the list. Test what happens when you try -2. Does it keep going? A lot of learning how to program is just doing stuff to see what happens.

```
for ml in my_list:
  print(ml)

for i, ml in enumerate(my_list):
  print("The {0}th element was {1}".format(i, ml))
```

```
1
2
3
4
5
6
7
8
9
The 0th element was 1
The 1th element was 2
The 2th element was 3
The 3th element was 4
The 4th element was 5
The 5th element was 6
The 6th element was 7
The 7th element was 8
The 8th element was 9
```

Conditionals in python

```
if 2 == 3:
  print("Wha.....?\n\n")
```

```
elif 3 == 2:
  print("Now that is odd")
else:
  print("2 does not equal 3.")
```

```
2 does not equal 3.
```

Note that we use colons and white space to organize code in python, not the {} of R.

> **!** Important
>
> Python uses a different syntax to define functions from R.

```
def my_add(x, y):
  return(x + y)
```

9.4 Popular and Useful Python Libraries

1. Numpy
2. Scipy
3. Matplotlib
4. Pillow
5. Sympy

Chapter 10

Programming Experiments

One of the most common uses of computers in psychology is for the programming of experiments. Classic experiments often involve a combination of visual stimuli that the participant has to respond to in someway. Typical responses are often button presses or mouse clicks, though many other varieties of stimuli and reports are used. While many programming languages can be used for this task we will begin with one of the most popular, Python. And if time permits we can explore some examples with Javascript as the world of experimental psychology is increasingly complementing the usual lab based testing paradigm with online studies.

10.1 All-in-one options

Excellent tools exist that provide turn-key solutions. A very popular one is Psychopy. This package is available for the popular operating systems and gives a gui like builder that allows you to program visually. It offers the opportunity to output your experiment into a web friendly format that can integrate with an online hosting service for performing experiments. However, my experience is that while such tools make it easy to get started they don't help people develop the general skills that will transfer into programming that distinctive experiment that no one else has done. And the knowledge you gain is often interface specific. It is not general, and so your learning cannot transfer into writing R code or porting your experiment to MATLAB if that becomes the tool a collaborator wants to use. Even if this is the sort of tool you eventually settle on for the very reasonable reasons that your experimental needs are not complex and the quality of the implementation allows you to get work done more easily and on board new investigators more quickly, it will still make you better at planning and handling problems if you have a knowledge of what is going on under the hood.

10.2 Python

The following examples focus on the sort of experiment where you want something to appear on the screen, and you want feedback from the participant. Current computer graphics don't work like they did twenty years ago when we all had heavy CRTs on our desks, but one can still deploy some of the intuitions of that era.

The monitor is a screen and it is displaying a picture. In the background what is to be shown next is in preparation and at some point what is on the display now will be replaced by the next collection in the queue. Even though we no longer literally raster the display from top to bottom and side to side you can still usefully think in terms of *frames*. Thus, almost all your experiments will need the mechanics of a window the participant views and in which you can write things you want them to see.

10.2.1 Pygame

For practice I will use the pygame python library. There are a large number of help and tutorials for this well established library that has been in use for decades. It is well vetted and documented, and relies on a simple direct library for video manipulations.

10.2.1.1 Some Links for Getting Started with Pygame

1. The pygame getting started wiki. Most of my examples will draw from this site.
2. Real python pygame tutorial
3. Game Development Academy

10.2.1.2 Installation

A virtual environment is recommended, but not necessary. If you are using a venv activate it first.

Then

```
pip install pygame                                    ①

python -m pygame.examples.aliens                      ②
```

① This line installs the library.
② This launches a test game. If you see the game, you have installed pygame. I find this game plays a little fast on modern hardware.

One of the better ways to get started is to gradually hand in functionality one step at a time until you have the basics of what you need on a trial and then to refine that to loop through the multiple trials you will probably need.

```
import sys,pygame
pygame.init()

mywindow = pygame.display.set_mode([800,800])
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
    mywindow.fill((128,15,15))
    pygame.display.flip()

pygame.quit()
sys.exit()
```

Figure 10.1: This python program will, assuming you have installed pygame, launch a window and close it when you click the "x". Can you change the window size and background color?

Here is an example of a bit of code that shows some elementary use of the mouse, response time, shape and text functionality.

```
from itertools import compress
import sys,pygame
pygame.init()


my_font = pygame.font.Font(None,72)
test_text = my_font.render("Hello P390", 1, (200,10,200))
test_text_pos = test_text.get_rect(centerx = 400)

#Helper functions
def which_buttons (bs):
    return(list(compress(range(len(bs)),bs)))

#button colors
clrs = [(0,0,255),(0,225,0),(225,0,0)]
actv_clr = 0
rslts = {'trial_num': -1,'color' : None ,'time' : None}

mywindow = pygame.display.set_mode([800,800])
running = True
myrt_clock = pygame.time.Clock()
t1 = myrt_clock.tick(60)
anew = pygame.mouse.get_pressed()
trl = -1,
```

```
clr = clrs[0]
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
    mywindow.fill((128,15,15))
    pygame.draw.circle(mywindow, clrs[actv_clr], (400,400), 75)
    mywindow.blit(test_text, test_text_pos)
    pygame.display.flip()
    aold = pygame.mouse.get_pressed()
    if(any(aold) and (aold != anew)):
        t2 = myrt_clock.tick(60)
        rslts['time'] = t2 - t1
        rslts['trial_num'] = rslts['trial_num'] + 1
        actv_clr = which_buttons(aold)[0]
        rslts['color'] = actv_clr
        anew = aold
        print(rslts)

pygame.quit()
sys.exit()
```

10.2.1.3 Next Steps

You will want to save your data. As a first step see if you can open a file, and the if you can use the dictionary created to add lines to a csv file.

Chapter 11

Handling Data

Before we can analyze data, we have to have some data. Broadly speaking our data may come from experiments we run ourselves, or it may come from data that is shared with us. In the first case we have some freedom to choose the format. In the later, we simply have to take what we are given. Therefore, we need to consider both how to save and convert data that we collect from an experiment. And we also need to be broadly familiar with common formats of data storage that others may use.

11.0.1 Common Data Formats And Some Advantages of Each

- Plain Text

  Plain text is easy to do and human readable. It usually will require a unique parser, and forces all items to be text. Thus, things like numbers lose their specificity, and things like lists can become painful to retrieve.

- CSV
- TSV

  CSV and TSV view all data as rows and a unique character (commas or tabs) separates the cells of the rows. Another character, typically a new line (often seen written as `\n`) is what distinguishes one row from the next. These file types are, largely, human readable if the number of columns and rows are not too large. They are not as space efficient as other formats, but that may be a reasonable tradeoff when the size of the data files is small. It can be a challenge with this format to deal with some sorts of data, because often numbers can be coerced to strings making the analysis more cumbersome.

- JSON

JSON stands for Java Script Object Notation. Although, as the name implies, it had its origin in javascript it is largely language agnostic and is a common format for the storage of data of all types. Its being used as a standard means it has wide support from many programming languages and that means there are good tools for exporting and storing for most programming languages. It can be quite flexible in the type of data stored. It is not friendly for writing by hand, but the use of tools makes it fairly easy to take experimentally collected data created in, say, a python dictionary and convert it to a json format.

- Pickle

  This is a binary format particular to the python programming language. It can be quicker and more efficient on storage space, but it is not human readable unless you convert it from the binary format to a textual format. It is also limited to the python world. If you choose this data storage format you are making a firm choice that you will be a python user, and so must your collaborators.

- Dat (R)

  This is an R data format. Like pickle it is binary and like pickle it is language specific, but rather than python the language is R. Maybe it is because I feel more comfortable doing data analysis in R, but I don't view this as hard to use as pickle. Usually my data is small enough to fit in a single workspace and I use the .dat format to save work at the end of a day, but I don't use it as my primary storage format when generating data experimentally.

11.0.2 A JSON Illustration

```
import random as r
import json as j
a = {'name': 'John Doe', 'age': 24, 'rt': r.random()}
js = j.dumps(a)
js
'{"name": "John Doe", "age": 24, "rt": 0.8463734340719113}'
```

Figure 11.1: The `dumps` command will output the json version as a string. Note this example is using python, but you can do the same thing with R. Thus JSON is more portable. Note there is a very similar python function without the terminal *s*, `json.dump`, that will save the json data to a file. Some other code is needed to create the file pointer. But once that is done, and the json data saved, you can read it back with `json.load()`

A JSON tutorial can be found here.

```
import json
a = {'name': 'John Doe', 'age': 24}
js = json.dumps(a)

# Open new json file if not exist it will create
fp = open('test.json', 'a')

# write to json file
fp.write(js)

# close the connection
fp.close()
```

Figure 11.2: This example was taken from [StackOverflow] shows a working example.

11.0.3 A Pickle Python Example

Pickling is a data serialization library specific to the python ecosystem. That makes it useful if you are coding your experiment in python, but not in any other language. However, it can be very efficient on space, and there is good support for it in the python language so many experimental coding libraries built around python use pickle for data storage.

This code from a GeeksforGeeks article provides a nice minimal working example.

```
import pickle

def storeData():
    # initializing data to be stored in db
    Omkar = {'key' : 'Omkar', 'name' : 'Omkar Pathak',
    'age' : 21, 'pay' : 40000}
    Jagdish = {'key' : 'Jagdish', 'name' : 'Jagdish Pathak',
    'age' : 50, 'pay' : 50000}

    # database
    db = {}
    db['Omkar'] = Omkar
    db['Jagdish'] = Jagdish

    # Its important to use binary mode
    dbfile = open('examplePickle', 'ab')

    # source, destination
    pickle.dump(db, dbfile)
    dbfile.close()
```

```
def loadData():
    # for reading also binary mode is important
    dbfile = open('examplePickle', 'rb')
    db = pickle.load(dbfile)
    for keys in db:
        print(keys, '=>', db[keys])
    dbfile.close()

if __name__ == '__main__':
    storeData()
    loadData()
```

Note that `__name__` at the end. That is a clever way to handle python files that allows you to use them as either a library or a script.

11.0.4 A Simple R Example

When working in the R environment the `save` and `load` function provide convenient ways to save your data and variables in their current state so that you can make available in your next interactive session what was available in your current one. However, this may not be the most interoperable way to work with your data if you need to share with collaborators who may not use R or if you yourself are open to the possibility of changing your analysis software. R offers two main functions for reading and writing data called `read` and `write`. Each of these has several specialization that target particular file types (e.g. `read.csv`) and also allow a number of customizations (e.g. whether to ignore a number of lines at the top of a file if there is a plain text header). Usually one of these variants will allow you to read your data into R or write data to a file that can be read by others.

> Class Question
>
> Can you figure out how to write R data to a json format? R has several built in data sets. A common one is `cars`. Can you write the `cars` data to a file storing it in json format?

```
data(cars)
library(jsonlite)
write_json(cars, "cars.json")
print(toJSON(head(cars,10), pretty = TRUE))
```

```
[
  {
    "speed": 4,
    "dist": 2
  },
  {
```

```
    "speed": 4,
    "dist": 10
  },
  {
    "speed": 7,
    "dist": 4
  },
  {
    "speed": 7,
    "dist": 22
  },
  {
    "speed": 8,
    "dist": 16
  },
  {
    "speed": 9,
    "dist": 10
  },
  {
    "speed": 10,
    "dist": 18
  },
  {
    "speed": 10,
    "dist": 26
  },
  {
    "speed": 10,
    "dist": 34
  },
  {
    "speed": 11,
    "dist": 17
  }
]
```

```
cars_from_json <- read_json("cars.json")
```

11.0.5 Considering Pandas and Interoperability with R

Pandas is a major python library and effort to provide cutting edge tools for statistics and analysis to match what is on offer elsewhere. In my experience they are not the first place that statisticians develop their cutting edge tools, R remains most popular, but for all else the choice between the two, R and Python, is really a matter of preference and comfort. Pandas is a very powerful tool

and if you are more likely to use python in the future you will want to explore Pandas fully.

Pandas can usually be installed with `pip install pandas`. Because of the size and complexity of python installations a virtual environment is recommended. If you are using `uv` and a virtual environment as recommended make sure you activate that environment and use prefix with `uv`.

Even if you are not thinking of doing your data analysis in python, if you are writing your experiments in python you might want to take advantage of some of the pandas tooling to make saving your data more convenient.

IAMHERE – need to test the following and fix the level of headers. – Britt Jan 6 2026

11.0.6 Pandas and R Dataframes Illustrated.

Since these two methods of handling data are so common we will run through a series of illustrative exercises that will teach you their differences and help you understand and compare their syntax.

11.1 Dataframe Self-Learning Exercises: R and Python Pandas

11.1.1 Prerequisites

- You have R installed and can run R code
- You have Python installed with pandas library (`pip install pandas` or `uv pip install pandas`)
- You can execute code in either RStudio, Jupyter notebooks, or your preferred IDE

11.1.2 Learning Philosophy

These exercises are designed to help you discover similarities and differences between R dataframes and Python pandas dataframes through hands-on experimentation. You'll work with the same dataset in both languages to build intuition about how each handles common data analysis tasks.

11.2 Part 1: R Dataframe Exercises

11.2.1 Exercise Set R1: Creating and Exploring Dataframes

11.2.1.1 R1.1 Create a Research Dataset

Task: Create a dataframe with experimental data.

```
# Create a dataframe with participant data
participants <- data.frame(
  subject_id = c(101, 102, 103, 104, 105, 106, 107, 108, 109, 110),
  age = c(22, 25, 23, 28, 21, 24, 26, 23, 27, 22),
  condition = c("control", "treatment", "control", "treatment", "control",
                "treatment", "control", "treatment", "control", "treatment"),
  reaction_time = c(450, 380, 470, 360, 490, 370, 460, 350, 480, 365),
  accuracy = c(0.85, 0.92, 0.83, 0.95, 0.80, 0.93, 0.84, 0.96, 0.82, 0.94)
)

# View the dataframe
print(participants)

# Check structure
str(participants)
```

Questions to answer: - How many rows and columns does your dataframe have? - What data types are in each column? - Is `condition` stored as a character or factor?

Expected outcome

You should see a dataframe with 10 rows and 5 columns. The `str()` function shows that `condition` is stored as a character vector by default in modern R. You can see the data types: numeric for age, reaction_time, and accuracy; character for condition; numeric for subject_id.

11.2.1.2 R1.2 Basic Dataframe Information

Task: Explore your dataframe's properties.

```
# Number of rows
nrow(participants)

# Number of columns
ncol(participants)

# Dimensions (rows, columns)
dim(participants)
```

```
# Column names
names(participants)
# or
colnames(participants)

# First few rows
head(participants)

# Last few rows
tail(participants)

# Summary statistics
summary(participants)
```

Questions to answer: - What does `summary()` tell you about each column? - How does it handle numeric vs. character columns differently?

Expected outcome

`summary()` provides descriptive statistics (min, max, mean, median, quartiles) for numeric columns and counts for character/factor columns. This is a quick way to check your data for outliers or data entry errors.

11.2.1.3 R1.3 Selecting Columns

Task: Practice different ways to access columns.

```
# Method 1: Dollar sign notation
participants$reaction_time

# Method 2: Square brackets with column name
participants[, "reaction_time"]

# Method 3: Square brackets with column number
participants[, 4]

# Select multiple columns
participants[, c("subject_id", "reaction_time", "accuracy")]

# Using subset to select columns
subset(participants, select = c(subject_id, reaction_time))
```

Questions to answer: - Which method do you find most readable? - What's the difference between `participants$reaction_time` and `participants[, "reaction_time"]`? - What happens if you use single brackets: `participants["reaction_time"]`?

Expected outcome

`$` returns a vector, while `[, "column"]` returns a vector by default but can return a dataframe if you use `drop=FALSE`. Single brackets `["column"]` returns a dataframe with one column. Understanding these differences is important for avoiding errors in your analysis code.

11.2.1.4 R1.4 Filtering Rows

Task: Select subsets of your data based on conditions.

```
# Filter for treatment condition only
treatment_group <- participants[participants$condition == "treatment", ]
print(treatment_group)

# Filter for participants younger than 25
young_participants <- participants[participants$age < 25, ]
print(young_participants)

# Multiple conditions: treatment AND age < 25
young_treatment <- participants[participants$condition == "treatment" &
                                participants$age < 25, ]
print(young_treatment)

# Using subset function (often more readable)
treatment_group2 <- subset(participants, condition == "treatment")
young_treatment2 <- subset(participants, condition == "treatment" & age < 25)
```

Questions to answer: - How many participants are in the treatment group? - How many young participants (age < 25) received treatment? - Which syntax do you find clearer: bracket notation or `subset()`?

Expected outcome

You should find 5 participants in treatment group, and 2 young participants in treatment. The `subset()` function is often more readable because you don't need to repeat the dataframe name and the comma notation is less error-prone.

11.2.1.5 R1.5 Computing Condition Means

Task: Calculate mean reaction time by condition.

```
# Method 1: Manual filtering
control_rt <- participants[participants$condition == "control", "reaction_time"]
treatment_rt <- participants[participants$condition == "treatment", "reaction_time"]

mean(control_rt)
mean(treatment_rt)

# Method 2: Using aggregate
aggregate(reaction_time ~ condition, data = participants, FUN = mean)
```

```
# Method 3: Using tapply
tapply(participants$reaction_time, participants$condition, mean)

# Get both mean and median
aggregate(reaction_time ~ condition, data = participants,
          FUN = function(x) c(mean = mean(x), median = median(x)))
```

Questions to answer: - Which condition has faster reaction times on average? - Which method do you find most intuitive? - What's the difference between the output formats of `aggregate()` and `tapply()`?

Expected outcome

Treatment group should have faster reaction times (lower values). `aggregate()` returns a dataframe which is often easier to work with, while `tapply()` returns a named vector. The formula syntax in `aggregate()` (`reaction_time ~ condition`) is very readable and common in R statistical functions.

11.2.1.6 R1.6 Adding and Modifying Columns

Task: Create new variables in your dataframe.

```
# Add a new column: convert RT from ms to seconds
participants$rt_seconds <- participants$reaction_time / 1000

# Create a categorical age group
participants$age_group <- ifelse(participants$age < 25, "young", "older")

# Create a performance score (combining RT and accuracy)
# Lower RT and higher accuracy = better performance
participants$performance <- participants$accuracy / (participants$reaction_time / 1000)

# View the updated dataframe
head(participants)

# Check the new columns
summary(participants)
```

Questions to answer: - How many columns does your dataframe have now? - What are the mean performance scores for each condition? - How would you create a three-level age group (young, middle, older)?

Expected outcome

Your dataframe now has 8 columns. You can calculate condition means for performance using the methods from R1.5. For three-level grouping, you'd use nested `ifelse()` or the `cut()` function.

11.2.1.7 R1.7 Basic Statistical Tests

Task: Perform a t-test comparing conditions.

```
# T-test: Does treatment affect reaction time?
t.test(reaction_time ~ condition, data = participants)

# Alternative: manual specification
control_rt <- participants$reaction_time[participants$condition == "control"]
treatment_rt <- participants$reaction_time[participants$condition == "treatment"]
t.test(control_rt, treatment_rt)

# Correlation: Age and reaction time
cor.test(participants$age, participants$reaction_time)

# Correlation: Reaction time and accuracy
cor.test(participants$reaction_time, participants$accuracy)
```

Questions to answer: - Is there a significant difference in reaction time between conditions? - What is the correlation between reaction time and accuracy? - What does a negative correlation between RT and accuracy suggest?

Expected outcome

You should see a significant difference between conditions (treatment faster). The correlation between RT and accuracy should be negative (faster responses = higher accuracy in this dataset), suggesting a speed-accuracy tradeoff is NOT present - participants who are faster are also more accurate.

11.3 Part 2: Python Pandas Dataframe Exercises

11.3.1 Exercise Set P1: Creating and Exploring Dataframes

11.3.1.1 P1.1 Create the Same Research Dataset

Task: Create an identical dataframe in pandas.

```
import pandas as pd
import numpy as np

# Create a dataframe with participant data
participants = pd.DataFrame({
    'subject_id': [101, 102, 103, 104, 105, 106, 107, 108, 109, 110],
    'age': [22, 25, 23, 28, 21, 24, 26, 23, 27, 22],
    'condition': ['control', 'treatment', 'control', 'treatment', 'control',
                  'treatment', 'control', 'treatment', 'control', 'treatment'],
```

```
    'reaction_time': [450, 380, 470, 360, 490, 370, 460, 350, 480, 365],
    'accuracy': [0.85, 0.92, 0.83, 0.95, 0.80, 0.93, 0.84, 0.96, 0.82, 0.94]
})

# View the dataframe
print(participants)

# Check structure
print(participants.info())
print(participants.dtypes)
```

Questions to answer: - How does the pandas output compare to R's `print()`? - What data types does pandas assign to each column? - How is `info()` similar to or different from R's `str()`?

Expected outcome

Pandas displays the dataframe with row indices (0-9) on the left. The `info()` method shows similar information to R's `str()`: column names, non-null counts, and data types. Pandas uses `int64`, `float64`, and `object` (for strings) as data types.

11.3.1.2 P1.2 Basic Dataframe Information

Task: Explore your dataframe's properties.

```
# Number of rows and columns
print(participants.shape)  # Returns (rows, columns)

# Number of rows
print(len(participants))

# Column names
print(participants.columns)

# First few rows
print(participants.head())

# Last few rows
print(participants.tail())

# Summary statistics
print(participants.describe())

# More detailed summary including non-numeric columns
print(participants.describe(include='all'))
```

Questions to answer: - How does `shape` compare to R's `dim()`? - What

does `describe()` show you? How is it different from R's `summary()`? - What happens when you use `describe(include='all')`?

Expected outcome

`shape` is a tuple (10, 5) similar to R's `dim()`. `describe()` shows statistics for numeric columns only by default, while R's `summary()` includes all columns. Using `include='all'` shows statistics for all column types, including count, unique values, and most frequent value for categorical data.

11.3.1.3 P1.3 Selecting Columns

Task: Practice different ways to access columns.

```
# Method 1: Bracket notation with column name
print(participants['reaction_time'])

# Method 2: Dot notation (only works if column name has no spaces/special chars)
print(participants.reaction_time)

# Select multiple columns - returns a dataframe
print(participants[['subject_id', 'reaction_time', 'accuracy']])

# Using loc for label-based selection
print(participants.loc[:, 'reaction_time'])

# Using iloc for position-based selection
print(participants.iloc[:, 3])  # 4th column (0-indexed)
```

Questions to answer: - What's the difference between `participants['reaction_time']` and `participants[['reaction_time']]`? - When would you use `loc` vs `iloc`? - How does pandas column selection compare to R's methods?

Expected outcome

Single brackets with one column name returns a Series (like a vector), double brackets returns a DataFrame. `loc` uses labels (column/row names), `iloc` uses integer positions. This is similar to R's distinction between `$` (returns vector) and `[, "col"]` (can return vector or dataframe).

11.3.1.4 P1.4 Filtering Rows

Task: Select subsets of your data based on conditions.

```
# Filter for treatment condition only
treatment_group = participants[participants['condition'] == 'treatment']
print(treatment_group)

# Filter for participants younger than 25
young_participants = participants[participants['age'] < 25]
```

```
print(young_participants)

# Multiple conditions: treatment AND age < 25
young_treatment = participants[(participants['condition'] == 'treatment') &
                               (participants['age'] < 25)]
print(young_treatment)

# Using query method (often more readable)
treatment_group2 = participants.query('condition == "treatment"')
young_treatment2 = participants.query('condition == "treatment" and age < 25')
print(young_treatment2)
```

Questions to answer: - Why do you need parentheses around each condition when using `&`? - How many participants are in the treatment group? - How does `query()` compare to R's `subset()`?

Expected outcome

Parentheses are needed because of operator precedence in Python. You should find 5 treatment participants and 2 young treatment participants. The `query()` method is similar to R's `subset()` in readability, using string expressions instead of boolean indexing.

11.3.1.5 P1.5 Computing Condition Means

Task: Calculate mean reaction time by condition.

```
# Method 1: Manual filtering
control_rt = participants[participants['condition'] == 'control']['reaction_time']
treatment_rt = participants[participants['condition'] == 'treatment']['reaction_time']

print(f"Control mean: {control_rt.mean()}")
print(f"Treatment mean: {treatment_rt.mean()}")

# Method 2: Using groupby
print(participants.groupby('condition')['reaction_time'].mean())

# Method 3: Using groupby with agg for multiple statistics
print(participants.groupby('condition')['reaction_time'].agg(['mean', 'median', 'std']))

# Multiple columns at once
print(participants.groupby('condition')[['reaction_time', 'accuracy']].mean())
```

Questions to answer: - Which condition has faster reaction times? - How does `groupby()` compare to R's `aggregate()` or `tapply()`? - What does the `agg()` method allow you to do?

Expected outcome

Treatment group has faster reaction times. `groupby()` is similar to R's `aggregate()` but often more flexible. The `agg()` method lets you apply multiple functions at once, returning a nice summary table. This is very powerful for exploratory data analysis.

11.3.1.6 P1.6 Adding and Modifying Columns

Task: Create new variables in your dataframe.

```
# Add a new column: convert RT from ms to seconds
participants['rt_seconds'] = participants['reaction_time'] / 1000

# Create a categorical age group
participants['age_group'] = participants['age'].apply(
    lambda x: 'young' if x < 25 else 'older'
)

# Alternative using numpy where (similar to ifelse)
participants['age_group2'] = np.where(participants['age'] < 25, 'young', 'older')

# Create a performance score
participants['performance'] = participants['accuracy'] / (participants['reaction_time'] / 1000)

# View the updated dataframe
print(participants.head())

# Check the new columns
print(participants.describe())
```

Questions to answer: - How many columns does your dataframe have now? - What's the difference between using `apply()` with a lambda vs `np.where()`? - How does adding columns in pandas compare to R?

Expected outcome

Your dataframe now has 9 columns (one more than R because we created age_group twice). `apply()` with lambda is more flexible but can be slower; `np.where()` is vectorized and faster, similar to R's `ifelse()`. Both languages make it easy to add columns with simple assignment.

11.3.1.7 P1.7 Basic Statistical Tests

Task: Perform a t-test comparing conditions.

```
from scipy import stats

# T-test: Does treatment affect reaction time?
control_rt = participants[participants['condition'] == 'control']['reaction_time']
treatment_rt = participants[participants['condition'] == 'treatment']['reaction_time']
```

```
t_stat, p_value = stats.ttest_ind(control_rt, treatment_rt)
print(f"T-test results: t={t_stat:.3f}, p={p_value:.4f}")

# Correlation: Age and reaction time
corr_age_rt, p_age_rt = stats.pearsonr(participants['age'], participants['reaction_time
print(f"Correlation (age, RT): r={corr_age_rt:.3f}, p={p_age_rt:.4f}")

# Correlation: Reaction time and accuracy
corr_rt_acc, p_rt_acc = stats.pearsonr(participants['reaction_time'], participants['acc
print(f"Correlation (RT, accuracy): r={corr_rt_acc:.3f}, p={p_rt_acc:.4f}")

# Using pandas corr() for correlation matrix
print(participants[['age', 'reaction_time', 'accuracy']].corr())
```

Questions to answer: - Are the statistical results the same as in R? - How does scipy's output format differ from R's? - What does the correlation matrix show you?

Expected outcome

Results should be identical to R (within rounding). Scipy returns tuples (statistic, p-value) rather than R's detailed output objects. The correlation matrix shows all pairwise correlations at once, which is useful for exploring relationships in your data.

11.4 Part 3: Comparative Exercises

11.4.1 Exercise C1: Side-by-Side Comparison

Task: Work through the same analysis in both languages and compare.

Scenario: You want to know if there's a relationship between age and performance, separately for each condition.

R Version:

```
# Calculate correlation for each condition
by(participants[, c("age", "performance")],
   participants$condition,
   function(x) cor(x$age, x$performance))

# Or using aggregate with a custom function
aggregate(cbind(age, performance) ~ condition,
          data = participants,
          FUN = function(x) cor(x[,1], x[,2]))
```

Python Version:

```python
# Calculate correlation for each condition
participants.groupby('condition').apply(
    lambda x: x['age'].corr(x['performance'])
)

# Alternative: manual calculation
for condition in participants['condition'].unique():
    subset = participants[participants['condition'] == condition]
    corr = subset['age'].corr(subset['performance'])
    print(f"{condition}: {corr:.3f}")
```

Questions to answer: - Which approach feels more natural to you? - How do the syntax patterns differ? - Which language makes this task easier?

Expected outcome

Both accomplish the same task but with different syntax. R's formula interface (`~ condition`) is very concise. Python's `groupby().apply()` is explicit about the grouping and function application. Neither is objectively "better" - it depends on your background and preferences.

11.4.2 Exercise C2: Data Reshaping Challenge

Task: Create a summary table showing mean RT and accuracy by condition.

R Version:

```r
# Using aggregate
summary_table <- aggregate(cbind(reaction_time, accuracy) ~ condition,
                           data = participants,
                           FUN = mean)
print(summary_table)

# Using dplyr (if available)
# library(dplyr)
# participants %>%
#   group_by(condition) %>%
#   summarise(mean_rt = mean(reaction_time),
#             mean_acc = mean(accuracy))
```

Python Version:

```python
# Using groupby with agg
summary_table = participants.groupby('condition')[['reaction_time', 'accuracy']].mean()
print(summary_table)

# Reset index to make condition a column instead of index
summary_table_reset = participants.groupby('condition')[['reaction_time', 'accuracy']].mean().res
```

```
print(summary_table_reset)

# Rename columns for clarity
summary_table_renamed = participants.groupby('condition').agg({
    'reaction_time': 'mean',
    'accuracy': 'mean'
}).rename(columns={'reaction_time': 'mean_rt', 'accuracy': 'mean_acc'}).reset_index()
print(summary_table_renamed)
```

Questions to answer: - How do the default outputs differ between R and pandas? - What is the pandas "index" and how does it compare to R's row names? - When would you use `reset_index()`?

Expected outcome

R returns a dataframe with condition as a regular column. Pandas makes condition the index by default. The index is similar to R's row names but more powerful. Use `reset_index()` when you want the grouping variable as a regular column for further analysis or export.

11.4.3 Exercise C3: Saving Your Work

Task: Save your dataframe to a CSV file in both languages.

R Version:

```
# Save to CSV
write.csv(participants, "participants_r.csv", row.names = FALSE)

# Read it back
participants_loaded <- read.csv("participants_r.csv")

# Check if it's the same
identical(participants, participants_loaded)
```

Python Version:

```
# Save to CSV
participants.to_csv("participants_python.csv", index=False)

# Read it back
participants_loaded = pd.read_csv("participants_python.csv")

# Check if it's the same
print(participants.equals(participants_loaded))

# Check what's different if not equal
print(participants.dtypes)
print(participants_loaded.dtypes)
```

Questions to answer: - Why do we use `row.names = FALSE` in R and `index=False` in pandas? - Are the loaded dataframes identical to the originals? - What happens to data types when you save and reload?

Expected outcome

Both languages can save/load CSV files easily. Setting `row.names/index = FALSE` prevents writing row numbers to the file. Data types might change slightly (e.g., integers might become floats) depending on the CSV format, but the data values should be the same.

11.5 Reflection Questions

After completing all exercises, consider:

1. **Syntax Patterns:**
   - R uses `$` for column access; pandas uses `[]` or `.`
   - R uses formula notation (`y ~ x`); pandas uses method chaining
   - Which feels more intuitive to you?
2. **Indexing:**
   - R uses 1-based indexing; Python uses 0-based indexing
   - R has row names; pandas has a more powerful index system
   - How does this affect your code?
3. **Function vs. Method:**
   - R: `mean(df$column)` - function applied to vector
   - Pandas: `df['column'].mean()` - method called on object
   - What are the advantages of each approach?
4. **Grouping Operations:**
   - R: `aggregate()`, `tapply()`, `by()`
   - Pandas: `groupby()` with method chaining
   - Which is more flexible? Which is more readable?
5. **Statistical Functions:**
   - R has statistics built into the base language
   - Python requires importing scipy
   - Does this matter for your workflow?

11.6 Summary: Key Similarities and Differences

11.6.1 Similarities

- Both have dataframe structures with rows and columns
- Both can filter, select, and transform data
- Both can compute grouped statistics

- Both can save/load common file formats
- Both have extensive statistical capabilities

11.6.2 Key Differences

Aspect	R	Pandas
Column access	`df$col` or `df[, "col"]`	`df['col']` or `df.col`
Multiple columns	`df[, c("a", "b")]`	`df[['a', 'b']]`
Filtering	`df[df$x > 5, ]`	`df[df['x'] > 5]`
Grouping	`aggregate(y ~ group, df, mean)`	`df.groupby('group')['y'].mean()`
Adding columns	`df$new <- value`	`df['new'] = value`
Method vs function	Functions: `mean(df$x)`	Methods: `df['x'].mean()`
Indexing	1-based	0-based
Missing values	`NA`	`NaN` or `None`

11.7 Next Steps

Once comfortable with these basics, explore:

R: - `dplyr` package for data manipulation (very popular) - `tidyr` for reshaping data - `ggplot2` for visualization - `data.table` for large datasets

Python Pandas: - Advanced groupby operations - Merging and joining dataframes - Time series functionality - Integration with scikit-learn for machine learning

Both: - Handling missing data - Working with dates and times - Merging datasets - Pivot tables and cross-tabulations

---

11.8 Common Pitfalls

11.8.1 R

- Forgetting the comma in `df[condition, ]`
- Factor vs. character confusion
- Recycling rules can cause unexpected behavior

11.8.2 Pandas

- Forgetting that many operations return a copy, not modifying in place
- Index alignment can cause unexpected results
- Chained indexing warnings (`df[df['x'] > 5]['y'] = 10` is problematic)

11.8.3 Both

- Not checking data types after loading from files
- Assuming missing values are handled the same way
- Not verifying results with small test cases first

11.8.4 Old School: Storing data as a csv file with Python.

But before we show a pandas method let's see a lower tech, more old school method.

It is possible to manually write a csv file in python using a print command where you explicitly coerce everything to a string and intercalate commas along the way. For example,

```
a = 1
b = "britt"
c = 2.3
mycolumnheadings = "a," + "b," + "c" + "\n"
mystring = str(a) + "," + b + "," + str(c) + "\n"
print(mycolumnheadings + mystring)
```

```
a,b,c
1,britt,2.3
```

shows how to create a string from one's data. It is complicated and error prone. It may work for a start or for a very small project, but it does not scale well. In this example I am just printing for display, but if this was being used to save experimental data we would want to use a file handler.

11.8.4.0.1 Opening and Closing a File in Python

```
a = 1
b = "britt"
c = 2.3
mycolumnheadings = "a," + "b," + "c" + "\n"
mystring = str(a) + "," + b + "," + str(c) + "\n"
with open('/home/britt/gitRepos/uwlooP390/p390/myfile.csv','w+') as f:
    f.write(mycolumnheadings)
    f.write(mystring)
```

Now you can see if you can read that csv into R.

```
mydata = as.data.frame(read.csv("/home/britt/gitRepos/uwlooP390/p390/myfile.csv"))
mydata
```

```
  a     b   c
1 1 britt 2.3
```

11.8.4.0.2 A Better Way: Dictionaries

Python dictionaries (which have their counterparts in most programming languages though they may be called something different) are a convenient way to store data where you can give it a meaningful name. For example, in an experiment with numerous trials and conditions you could have a *key* in your dictionary for "trial no." and another for "condition". The *values* for these would then be updated as your experiment unfolded.

A first pass to using dictionaries is to blend this with a python library for CSV files. Then you can create the column headings from your keys and then each trial that you loop through you can use the dictionary to write a new row to your csv file. Some of the functions you might want to use are:

- `csv.DictWriter`
- `writeheader`
- `writerows`

If you set up and define your dictionary at the start of the file you only need to update the changing values as they occur.

Another approach that will start you getting some familiarity with python pandas library is to make to make a distinct dictionary for each of the trials in your experiment. Then you can concatenate these dictionaries to a list and use that list to create a pandas dataframe. That could then be used in analysis python code or for importing into R. Here is a short example to illustrate the principle.

```
import random as r
import pandas as pd

mylistofkeys=["name","v1","v2","v3","v4"]
```

```
mydictlist = []
for i in range(5):
    mydictlist.append({key:value for key,value in zip(["name","v1","v2","v3","v4"],["britt"] + [r
```

```
df = pd.DataFrame(mydictlist)
```

```
df.loc[:,'v3'].mean()
```

```
np.float64(0.5907036839939771)
```

```
myfilename = "mypandastest.csv"
df.to_csv(myfilename)
```

You can save pandas dataframes as a pickle file, but then you will not be able to easily read it into R. CSV remains the easiest and most direct way to do this. It is probably all you need for most use cases. Though there are other options.

```
rdf = read.csv("mypandastest.csv")
mean(rdf$v3)
```

```
[1] 0.5907037
```

11.8.4.1 Some Analysis Tools

Chapter 12

Lab Notebooks

While often associated with "wet labs" (chemistry/biology), a lab notebook is essential in Psychology to ensure **reproducibility**. You need to track not just your results, but the decisions you made while cleaning data, the specific parameters used in an analysis, and the dates files were modified.

Here are two ways to approach this in a computational environment.

12.0.1 Exercise 1: The Plain Text Log

The simplest digital notebook is a plain text file. It is future-proof, searchable, and works on every computer.

Try this in your terminal:

1. Create a file named `research_log.txt`.
2. Append the current date and a note about what you are doing.
3. Read the file back.

```
# 1. Create/Append the date to the file
date >> research_log.txt

# 2. Append a note
echo "Today I learned how to append text to a file." >> research_log.txt

# 3. Read the log
cat research_log.txt
```

12.0.2 Exercise 2: Jupyter Notebooks

Modern research often combines the "Lab Notebook" with the "Statistical Analysis" using tools like Jupyter Notebooks. These allow you to mix narrative

text, code, and results in a single document.

Try this in your terminal:

> ⚠ Warning
>
> The path to your virtual environment may be different. Adjust the line in the next section accordingly.

1. **Activate your virtual environment and install Jupyter:**

   ```
   source venv/bin/activate

   pip install jupyter ipykernel
   ```

2. **Launch Jupyter Notebook:**

   ```
   jupyter notebook
   ```

   This will open your browser to the Jupyter interface.

3. **Create a new notebook:** Click "New" → "Python 3" in the browser.

4. **Add narrative text:** In the first cell, change the dropdown from "Code" to "Markdown". Type:

   ```
   # Lab Session: Pilot Data Analysis
   Calculating the mean of my pilot data collected on [today's date].
   ```

   Press Shift+Enter to run the cell.

5. **Add code:** In the next cell (which should be "Code" by default), type:

   ```
   data = [200, 350, 410, 320]

   mean_rt = sum(data) / len(data)

   print(f"Mean reaction time: {mean_rt} ms")
   ```

   Press Shift+Enter to run the cell.

6. **Save your work:** Press Ctrl+S or go to File → Save.

You now have documentation and calculation in one place that you can share or return to later.

12.0.3 Further Reading

If you want to dive deeper into best practices for digital record keeping:

- Jupyter notebooks in Psychology
- 10 rules for lab notebooks (electronic)

- How to keep a lab notebook
- Pick an electronic lab notebook
- Lab notebook 2023 guide

12.0.4 An example

See Chapter 13 for an example of how a lab notebook in Quarto might look and be used. In the upper portion the use of the notebook for recording analyses and important project metadata, as well as interim data analyses. The second portion shows the use of a chronological lab notebook. This makes the notebook a living, dynamic document that helps you remember important details and records them in case later you need to find the documentation or date for an important event. These two styles are complementary anc can be used together.

Chapter 13

An Example Lab Notebook

```
---
# 1. METADATA: THE OFFICIAL RECORD
# (Purpose: This section contains essential information for identifying, tracking, and citing the
title: "PSYC [Course Number] Lab Notebook Entry: [Insert Activity Title]"
author: "Student Name(s) and ID(s)"
date: today # Automatically uses the current date. This serves as the official timestamp for the
# output:
#   html:
#     toc: true # Creates a Table of Contents for easy navigation
#     embed-resources: true # Makes the final HTML file self-contained
#   pdf: default # Uncomment if you prefer a PDF output
---
```

13.1 1. Research Aim & Hypotheses

13.1.1 Research Question

What is the effect of [**Independent Variable (IV)**] on [**Dependent Variable (DV)**] in our sample of [Population]?

13.1.2 Hypotheses

- **H1 (Alternative):** We predict that [Specific prediction based on IV and DV].
- **H0 (Null):** There will be no significant effect of the IV on the DV.

13.2 2. Methodology & Procedure

13.2.1 Participants

- **Recruitment Method:** [e.g., SONA, convenience sample]
- **Target Sample Size:** [Number]
- **Demographic Criteria:** [e.g., Age 18-25, native English speakers]

13.2.2 Materials & Apparatus

- **Psychological Measure/Scale:** [e.g., State-Trait Anxiety Inventory (STAI)]
- **Stimuli Details:** [e.g., 20 high-arousal images from the IAPS database]
- **Software/Equipment:** [e.g., PsychoPy, Qualtrics, R Studio]

13.2.3 Step-by-Step Procedure

1. [Detailed action 1, e.g., Calibrated the response box.]
2. [Detailed action 2, e.g., Participant received standardized instructions.]
3. [Detailed action 3, e.g., Ran 4 blocks of 20 trials each, with a 30-second break between blocks.]

13.2.4 Critical Observations & Deviations (The "Activity Log")

Observation 1 (Time: 10:45 AM): The internet connection briefly dropped for 3 minutes during the survey administration for Participant 7. The system appears to have saved their progress, but a note was made to check the final file for completeness.

Deviation: We were supposed to collect 20 participants, but due to time, only collected 18. Analysis must account for lower statistical power.

13.3 3. Data Processing & Analysis

13.3.1 3.01 Generating Fake Data

This is **not** part of a lab notebook. This is some code to generate random data so that the rest of the code can be tested.

```
# Lab Notebook Data Generation Script
# Purpose: Generates a sample CSV file with clean, pre-aggregated data
#          for use in the psychology lab notebook lesson (Quarto/R).

# Load necessary library
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.1.4     v readr     2.1.6
v forcats   1.0.1     v stringr   1.6.0
v ggplot2   4.0.1     v tibble    3.3.1
v lubridate 1.9.4     v tidyr     1.3.2
v purrr     1.2.1
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become er
```

```r
# --- Define Parameters ---
N_PARTICIPANTS <- 25
CONDITIONS <- c("Treatment", "Control")

# Parameters for Accuracy (The Dependent Variable)
MEAN_TREATMENT_ACC <- 0.85
MEAN_CONTROL_ACC <- 0.75
SD_ACCURACY <- 0.10

# Parameters for Reaction Time (The Filtering Variable)
MEAN_RT <- 650 # in milliseconds
SD_RT <- 150 # in milliseconds

# --- Generate the Data ---
set.seed(42) # Set a seed for reproducible random data

sample_raw_data <- tibble(
  participant_id = 1:N_PARTICIPANTS,
  condition = factor(rep(CONDITIONS, length.out = N_PARTICIPANTS)),

  # Filtering Column: 90% pass (1), 10% fail (0).
  attention_check = rbinom(N_PARTICIPANTS, 1, 0.9),

  # 1. ACCURACY Column (Dependent Variable, continuous score 0.0 to 1.0)
  # This column simulates the final aggregated accuracy score.
  accuracy = if_else(
    condition == "Treatment",
    rnorm(N_PARTICIPANTS, mean = MEAN_TREATMENT_ACC, sd = SD_ACCURACY),
    rnorm(N_PARTICIPANTS, mean = MEAN_CONTROL_ACC, sd = SD_ACCURACY)
  ) |> pmin(1.0) |> pmax(0.0), # Cap values between 0 and 1

  # 2. RT Column (Filtering Variable, in milliseconds)
  # This column simulates the mean reaction time for filtering.
  rt = round(rnorm(N_PARTICIPANTS, mean = MEAN_RT, sd = SD_RT))
)
```

```
# Introduce simulated outliers to test the filtering step in the notebook
# Participant 5 is too slow (tests RT filter)
sample_raw_data$rt[5] <- 3200
# Participant 10 fails attention check and is too slow (tests both filters)
sample_raw_data$rt[10] <- 4500
sample_raw_data$attention_check[10] <- 0

# --- Save the Data ---
# Create the 'data' directory if it doesn't exist (matching the template's path)
if (!dir.exists("data")) {
  dir.create("data")
}

# Save the dataset as a CSV file
write_csv(sample_raw_data, "data/psych_exp_raw_data_2025-11-01.csv")

# Final check of the data structure
print("Final sample data file created successfully in the 'data' folder.")
```

```
[1] "Final sample data file created successfully in the 'data' folder."
```

```
print(head(sample_raw_data))
```

```
# A tibble: 6 x 5
  participant_id condition attention_check accuracy    rt
           <int> <fct>               <dbl>    <dbl> <dbl>
1              1 Treatment               0    0.854   610
2              2 Control                 0    0.718   917
3              3 Treatment               1    0.948   739
4              4 Control                 1    0.650   689
5              5 Treatment               1    0.898  3200
6              6 Control                 1    0.766   680
```

13.3.2 3.1 Setup and Data Import

```
# Load necessary packages (e.g., for data manipulation, statistics)

raw_data <- read_csv("data/psych_exp_raw_data_2025-11-01.csv")
```

```
Rows: 25 Columns: 5
-- Column specification --------------------------------------------------------
Delimiter: ","
chr (1): condition
dbl (4): participant_id, attention_check, accuracy, rt

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
# Display the dimensions to check if import was successful
dim(raw_data)
```

```
[1] 25  5
```

```r
# Filter out participants who failed an attention check (Reaction Time > 3000ms)
cleaned_data <- raw_data |>
  filter(rt < 3000) |>
  # Calculate the primary dependent variable (mean accuracy across all trials)
  mutate(mean_accuracy = mean(accuracy))

# Value: Always check the final sample size after filtering.
n_final <- nrow(cleaned_data)
cat("Final sample size used for analysis after exclusions: ", n_final, "\n")
```

```
Final sample size used for analysis after exclusions:  23
```

13.3.3 Example: Run a two-sample t-test to compare mean accuracy between two conditions (Treatment vs. Control)

```r
t_test_result <- t.test(accuracy ~ condition, data = cleaned_data)

# Print the full results for comprehensive record-keeping
t_test_result
```

```

    Welch Two Sample t-test

data:  accuracy by condition
t = -3.6113, df = 19.462, p-value = 0.001803
alternative hypothesis: true difference in means between group Control and group Treatment is not
95 percent confidence interval:
 -0.23821132 -0.06358062
sample estimates:
  mean in group Control mean in group Treatment
              0.7119613               0.8628572
```

13.3.4 Example: Generate a plot of the primary finding

```r
cleaned_data |>
  group_by(condition) |>
  summarize(mean_acc = mean(accuracy), se = sd(accuracy)/sqrt(n())) |>
  ggplot(aes(x = condition, y = mean_acc)) +
  geom_bar(stat = "identity", fill = "skyblue") +
  geom_errorbar(aes(ymin = mean_acc - se, ymax = mean_acc + se), width = 0.2) +
  labs(y = "Mean Accuracy", x = "Experimental Condition") +
  theme_minimal()
```

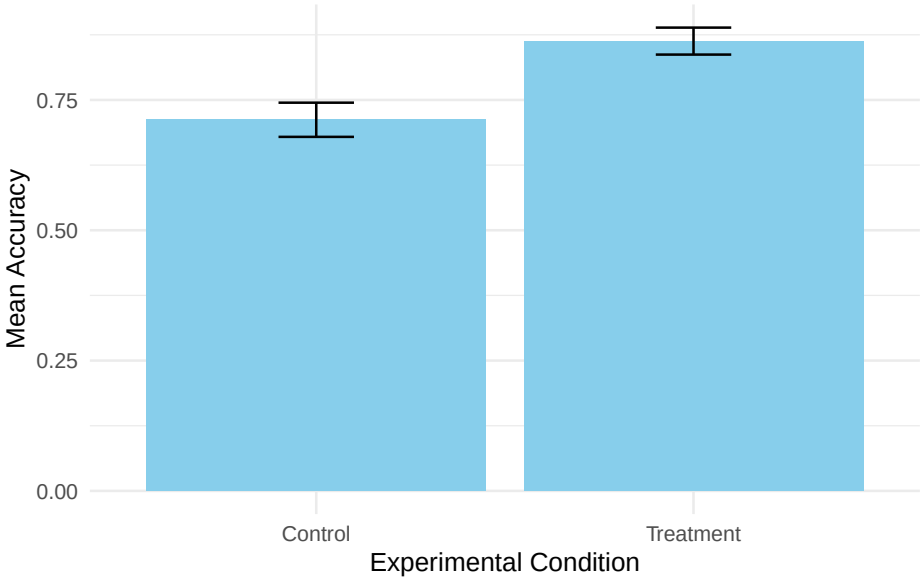


Figure 13.1: Mean Accuracy Scores by Condition (Treatment vs. Control)

13.4 4. Project Chronology and Daily Log

*(Purpose: This section serves as the **scientific diary** for the project. Every day you work on the research—whether collecting data, cleaning code, or reading literature—you must log your activities, thoughts, and any problems encountered. This maintains the chain of custody and the historical record of the research.)*

13.4.1 4.1 Log Entry: 2025-11-04 (Data Collection Session 2)

- **Time:** 1:00 PM - 4:00 PM.
- **Activity:** Collected data from 8 new participants (P19 - P26) using the same protocol as the first session.
- **Observations:** The data collection went smoothly, but **Participant 22** reported feeling tired and may have rushed the final block. No technical errors.
- **New Files Generated:** `raw_data_2025-11-04.csv`.
- **Action Required:** Merge `raw_data_2025-11-04.csv` with the existing dataset before the next analysis run.

13.4.2 4.2 Log Entry: 2025-11-05 (Literature Review & Code Cleanup)

- **Time:** 9:00 AM - 11:30 AM.

- **Activity:** Reviewed literature on the IV. Found a key paper suggesting the effect is stronger in female participants.
- **Thought/Reflection:** *I should add a gender variable to the final model to check for this interaction effect (a moderation analysis).*
- **Code Activity:** Renamed the `cleaned_data` object in the analysis script to `data_for_t_test` for better clarity.
- **Action Required:** Plan the code structure for running an **ANOVA** with gender as an additional factor next week.

13.4.3 4.3 Log Entry: 2025-11-10 (Initial Analysis Run & Troubleshooting)

- **Time:** 1:30 PM - 2:30 PM.
- **Activity:** Ran the `main-analysis` chunk in the Quarto notebook (Section 3.3) on the full dataset (P1 - P26).
- **Problem:** The `t.test()` code returned an error: `"Error: variable lengths differ."`
- **Resolution:** Identified that the error was caused by a participant's row being deleted in the **cleaning** step without the corresponding row being deleted in the original R object. The issue was resolved by reloading `raw_data` inside the `setup-and-import` chunk, ensuring the entire workflow is self-contained and reproducible.
- **Conclusion:** The p-value for the main effect is $p = 0.08$. The effect is trending, but not significant. More data is required.

Chapter 14

Writing Scientific Documents

Writing scientific documents is more than writing papers. It can also include posters, websites, books, and all of these may require images, graphs, references, formuals, and even code.

One way the computer eases this process is by allowing us to write in a more familiar language and then letting the computer take over the process of exporting the more verbose and complex text necessary to produce the final version, which might, for example, be a webpage or pdf.

The pieces of this pathway that we will use are a languge for writing, often lumped under the generic heading of markup languages, a program for converting our markup into an intermediate language that looks more like a programming language, and a final processor, again a computer program, that will generate the final document from our intermediate representation.

While we can write our initial markup version on any text editor, if we use a tool intended for this use it can help us avoid typos and having to look up lots of details. It can take our initial few characters and prompt us for the correct contintuations and completions. Thus IDEs can also make very effective writing tools.

The languages we will use for text generation are first LaTeX and second Markdown. The first is used primarily for typesetting mathematics and code. The latter can also be used for that, requires less work to get going, but is still, for the moment, somewhat less comprehensive in the complexity of documents it can render.

14.1 IDEs and Writing up your Research

Class Question

What does IDE stand for and what are examples of IDEs?

14.1.1 VSCode

We have been using VS Code in this course (see Section 3.1.1 if you still need to install and learn the basics).

14.1.2 Overleaf

VS Code is far from your only option. For technical writing and collaboration many scientists rely on Overleaf.

The university provides you with a full subscription to Overleaf. Overleaf allows you to use LaTeX like you might use a Google Doc. You can collaboratively write and edit with collaborators. In the last year I have used Overleaf to write two grant proposals with collaborators. Because Overleaf is LaTeX focused it has a number of helpful features to give you hints while you write and templates to get you started.

LaTeX and Knitr in Overleaf][knitr]] with overleaf. A scientific poster with Overleaf?

Beamer is a presentation mode that produces pdf slides from a LaTeX source.

Overleaf has a beginners tutorial.

Nature will [let you submit your manuscript](https://www.nature.com/srep/author-instructions/submission-guidelines) from an Overleaf template.

14.1.3 RStudio

RStudio is a popular IDE for writing R code. R is a programming language particularly useful for statistical analyses. While initially RStudio was only for the R language the number of programming languages it currently supports has grown. It still is not as widely useful as VS Code, and many of RStudio's features are designed for the interactive evaluation of data, and so it has less of the cutting edge features that software developers expect in their code tools. Whether you want to use RStudio or not will depend in part on your planned activity.

14.1.4 Others

These are far from the only writing tools that allow for an integrated writing and data analysis experience. My own favorite tool is Emacs, which gives you

an ide you can program. It has a rather steep learning curve.

14.2 Intermediate Languages

Once you have a tool where you can write. You will need to write in a language. This means that in addition to the usual text you are writing you will use particular symbols, keywords and punctuation that will mean something to a processing program when you export your document from the text editor or IDE. For instance, in LaTeX `\emph{What?}` would lead the word "What?" being emphasized on export. In markdown similar output would come from writing `*What?*`.

14.2.1 LaTeX

We will not be using LaTeX as our primary markup language since it does not natively support the ability to evaluate code inline. This functionality can be added by following a multistage process where code blocks are marked and a first pass executes the code and inserts the output into the .tex file. Then the .tex file can be processed as usual. Given the complexity of this work flow it is not necessary or advised for beginners. A general discussion of this approach though, for those interested, can be found by reading more about noweb.

While LaTeX is more than we need to get started it is one of the oldest typesetting languages, and therefore many more modern languages have inherited some of its conventions and use its syntax for writing mathematical equations. Thus, knowing some LaTeX basics is highly desirable.

14.2.2 Markdown

Markdown began as a simple, if punny, markup language. Initially it was simply text, and was intended for exporting to html, the language of web pages.

Instead of having to insert a lot of `<section> ... </section>` type anchors in your writing you could simply do `# My section` and when exporting from .md to .html the computer would bracket the text following the hash (`#`) with the appropriate anchors. From this humble beginning very complex systems have emerged that make it much easier for all of us to write documents that export to webpages, pdf, and more. Qmd is one modern variant.

Additional Reading

- LaTeX
  - LaTeX
- A guide to LaTex for Word Users (pdf)
- The descendants of markdown
- Quarto/Qmd
  - Quarto

- Org Mode
  - Org Mode works particularly well with Emacs and allows you to embed code from multiple languages.

14.3 Things You Can Do With Quarto and Qmd files

Here are some of the helper links for things we can do with Quarto:

- journal formats try the Elsevier format for this course. A lot of psych journals are Elsevier owned.
- presentation
- Talking to databases.
- Make a website/blog for your work or lab
- And more ...

14.3.1 Blog Project in Quarto

In Chapter 6 we created a basic academic website with Quarto. In this variant you can make a blog.

The advantage of managing your blog in Quarto is that you get to take advantage of all the markdown language features. This is much more convenient than writing in html directly. You can also embed code and output just as you have been doing when you export your R code to html.

14.3.2 Creating a Blog with Quarto and GitHub Pages

While an academic website presents a polished professional identity, a blog serves a different purpose: regular, less formal expression of ideas, reflections on research, or documentation of learning. This tutorial builds on the website deployment skills you've already developed, but configures Quarto as a blog platform instead.

14.3.2.1 Why a Separate Blog Site?

Github limits you to one github pages website, but we don't want to overwrite the one you already made. The solution is to create a new "project" that also has a github-pages output, and then it will show up under your github name. That is why I can have a website at brittAnderson.github.io and put these course notes as brittanderson.github.io/uwlooP390.

This separation also allows you to keep a clean, professional look for your public facing personal site, and also other sites that are more project specific or experimental.

14.3.2.2 Creating the Blog Repository

Unlike your academic website which required the specific name `username.github.io`, a project site can have any repository name. We'll use `blog` for clarity.

1. **Create a new repository** on GitHub named `blog` (not `username.github.io`). Make it public.
2. **Clone the repository** to your local machine:

   ```
   git clone https://github.com/username/blog.git
   cd blog
   ```

3. **Initialize a Quarto blog project**: Note that we will be using a specific keyword here to clue quarto into the correct template. We are making a *blog* not a *website*. At least as far as Quarto is concerned. We know that a blog is a website.

   ```
   quarto create project blog .
   ```

   Choose "blog" when prompted, and confirm overwriting the directory.

4. **Preview your blog locally**:

   ```
   quarto preview
   ```

   Your browser should open displaying the default Quarto blog with sample posts.

14.3.2.3 Understanding Blog vs. Website Structure

The key architectural difference between a blog and a website is content organization:

Website (your academic site): - Static pages at the root level (`index.qmd`, `about.qmd`, `cv.qmd`) - Content rarely changes - Navigation via navbar links

Blog (this project): - A `posts/` directory containing individual posts - Each post lives in its own subfolder: `posts/post-name/index.qmd` - Homepage automatically generates a listing of posts - Posts have metadata (date, author, categories) that enables automation

14.3.2.4 Configuring for GitHub Pages

Because this is a project site (not your user site), Quarto needs to know the base URL to generate correct links.

Be careful here. This is of "type" blog, but you will have a section "website."

1. **Edit `_quarto.yml`** to add the site URL and configure output:

   ```
   project:
     type: blog
     output-dir: docs
   ```

```yaml
website:
  site-url: "https://username.github.io/blog"
  title: "Your Blog Title"
  description: "A place for thinking out loud"
  navbar:
    left:
      - text: "Home"
        file: index.qmd
      - text: "About"
        file: about.qmd
```

Replace `username` with your actual GitHub username. The `site-url` field ensures RSS feeds and internal links use the correct base path (`/blog`).

2. **Customize the about page** by editing `about.qmd`:

```yaml
---
title: "About This Blog"
---

[Explain what you plan to write about, why you're maintaining this blog, or what r
```

14.3.2.5 Creating Blog Posts

Each post is a separate Quarto document with structured metadata. The metadata isn't just decorative—it enables automatic listing pages, RSS feeds, and category filtering.

1. **Create a new post directory**:

```
mkdir -p posts/first-post
```

2. **Create the post file** `posts/first-post/index.qmd`:

```yaml
---
title: "My First Blog Post"
author: "Your Name"
date: "2024-01-15"
categories: [thoughts, meta]
---

## Why I'm Starting This Blog

[Your content here]

This is a space for less polished thinking than my academic work requires. I plan
```

3. **Preview the post**:

```
quarto preview
```

Navigate to your new post from the homepage listing.

The homepage (`index.qmd`) automatically generates a listing of all posts in the `posts/` directory, sorted by date. You don't need to manually maintain this list—Quarto reads the YAML metadata from each post.

14.3.2.6 Deployment

The deployment process is identical to your academic website:

1. **Build the site**:

   ```
   quarto render
   ```

2. **Commit and push**:

   ```
   git add .
   git commit -m "Initial blog setup"
   git push
   ```

3. **Configure GitHub Pages**: Navigate to repository Settings → Pages → Source: Deploy from branch → Branch: main → Folder: /docs → Save

Your blog will be available at `https://username.github.io/blog` within a few minutes.

14.3.2.7 Exercise

Create three blog posts with different metadata configurations to understand how Quarto uses this information:

1. **A post without categories**: Observe that it still appears in the listing but has no category tags.

2. **Two posts with overlapping categories**: Create posts tagged with `[research, methods]` and `[research, teaching]`. Notice how the category system creates implicit connections between posts.

3. **Posts with different dates**: Create posts dated a week apart. Verify that the listing page automatically orders them chronologically.

After creating each post, run `quarto preview` to see how the homepage listing updates automatically.

Why the `/docs` output directory? GitHub Pages expects site files in either the repository root or a `/docs` folder. Using `/docs` keeps your source files (`.qmd`) separate from generated HTML, making the repository more maintainable.

Why `site-url` matters: Without this field, Quarto assumes your site deploys at the domain root. For a project site at a subpath (`/blog`), links would break.

The `site-url` tells Quarto to prepend `/blog` to all internal links and configure RSS feeds correctly.

Directory structure philosophy: Each post in its own folder allows you to keep related files (images, data files) co-located with the post content. This modular organization scales better than dumping everything in a single directory.

14.3.3 Journal Article Authoring

Classroom Exercise and HW

Write an article with quarto/qmd and using any of the linked article formats. Each has instructions on their use. Your article should look similar to a real manuscript when you compile it to pdf. It should have all the components of an article, at least one figure, and at least one reference. You can use some of the example code in these notes for ideas so that you can a section of embedded code that creates a figure.
Before you begin you will want to make sure you have a LaTeX system installed. You will find some instructions in the operating system specific appendices, but for convenience a short instruction is to:
For the WSL people (in your WSL/Ubuntu terminal)

```
sudo apt update
sudo apt install texlive-full
```

And for the Mac people

```
brew install --cask mactex
# Add to PATH:
echo 'export PATH="/Library/TeX/texbin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

I find this easier to do from a terminal.

```
#| eval: false

quarto use template quarto-journals/elsevier
```

This will result in several questions for you to answer. When you are done you have starter versions of all the pieces you will need to get going.

Note Bene (Jan 2026). I ran into a bug when trying to render. You will need to navigate to the `./_extensions/quarto-journals/elsevier/elsarticle.cls` and delete that first `%` on line 1368. You only need to do this if you get an error that talks about `bibsection` and `bmsection`. See this issue

14.3.4 Presentations

Presentations are a standard component of professional life. Despite that importance the materials are often developed in a casual way that does not give the best

impact to the material. There are many more versatile tools than contemporary Power Point. Though power point can make an effective presentation its default use does not typically achieve that goal. Quarto provides easy access to two other presentation tools that will expand one's recognition of what an effective presentation tool can do.

14.3.4.1 Reveal.JS

Reveal.js is a javascript library for the creating presentations that work as well on the web as they do in person. Quarto also supports Reveal.js.

14.3.4.2 Beamer

Beamer is a package for LaTeX that allows professional quality typesetting. It is particularly relevant for presentations that have a lot of mathematical material, though it is quite capable of supporting any general presentation as well. It is less flexible for the web since it produces PDFs. These can be shared on the web, but they require the intermediate step of downloading them rather than just playing them. Beamer also has a "Poster" variant that can be used to make high quality academic posters.

Quarto also supports Beamer.

14.3.5 Academic Posters

Typest

Scientific Posters with Quarto

Chapter 15

Knowledge Management

This section deals with the issue of how you keep track of the information you discover and how you mine it for new ideas, associations, and connections. In this section we will consider the notion of the *zettlekasten* and some modern tools for emulating this technique.

For many contemporary adherents to the zettlekasten system their pole star is Niklas Luhman a social systems theorist who was an prolific publisher and who credited much of his fecundity to this technique. Essentially, the method is one of cross-referenced index cards, and historical precedents are easy to find.

Although there are different flavors to this "slip box" approach the consistent theme is that one reflect on their reading, summarize it, and connect it to other works and concepts in their data base. We can perhaps do this more easily [1].

15.1 (Zettlekasten)

1. Org-roam

Org-roam is a tool for use within the Emacs ecosystem. Emacs is, to paraphrase the old joke, an operating system managing as a text editor. Emacs comes with a suite of functions (`org-mode`) built in that facilitate outlining and note taking (and many more things besides). Org-roam is an additional piece of software designed to interact with the Emacs/Orgmode combination to implement many zettlekasten features. It is an emulation, in Emacs, of RoamResearch a self-described tool for networked thought.

The online manual for org-roam provides a brief introduction and overview of the tool. Very, perhaps too, briefly you use various key commands to create nodes

[1] It is a question whether the enhanced digital ease impacts effectiveness since key to the practice is the reading and re-reading of "cards" in the index.

in your network and link them to others. You include notes on entries in your bibliographic system, or free standing notes organized around particular concepts. These links can be viewed both forward (things they link to) or backward (things that link to them), and there are also graphical views. A brief demonstration of some of these tools will be provided in class.

2. Dendron - works with VS Code

This note-taking knowledge management tool is included here because it can interact well with the VS Code and Markdown approach we have been learning about through our use of Quarto.

One of the common features you will find for all these tools is the paring back of dependencies. Most rely on some sort of plain text system at their base. This means you can always just read your entries even if the fancy link and processing features break or prove unwieldy. This generally future proofs your "cards" and also often means that things can work faster and more robustly because there are fewer dependencies.

3. Obsidian

Obsidian is another very popular offering in the same space as the above. This one however will cost you money. I include it here because of its wide popularity. The overview page of the website provides a good general visual orientation to the offerings and features.

Classroom Exercise

Taking Dendron for a test drive.

1. Install Dendron from the VS Code Marketplace.
2. Fill out the survey and any extensions you want (I took all the recommended) and proceed to the tutorial.
3. This should like quite familiar to your. The `qmd` files we have been writing are really just augmented versions of this `md` format.
4. Use the terminal to figure out what directory your notes are being stored in.
5. Use Ctrl-L to create a new note then navigate back to the tutorial.
6. The authors keep talking about `vim`. Look up what `vim` is. Do you have `vim` or its more difficult cousin `vi` on your computer? Try to open one or the other in a terminal.
7. How do you creat a note hierarchy?
8. View the tree mode.
9. Use the Command Palette to view all the Dendron related commands, and at least read the next steps.

15.2 Reference Management

In this section we will look at tools for two important functions related to references for the papers we write and the research we do. How do we find the details for references that we may need to cite, and how do we ease the task of writing so that our references and citations are properly formatted and easy to update when we need to change the citation format for a manuscript.

15.2.1 Finding Citations

- Semantic Scholar

  This tool is supposed to help you find related papers. I have not found it all that helpful in my daily use. Do any of you have success stories to share?

- OpenAlex Try finding my articles here, and see if you can find out what my opinion on *attention* is.

- Google Scholar

  See if you can find any research articles on Albert Einstein's brain by Dr. Sandra Witelson of McMaster University. What I like about this tool is that you can also search for articles that cite particular articles. How many of the articles that cite Dr. Witelson also contain the word myelin?

- Consensus

  This is a new AI Driven search engine. It has some very nice features, but do be careful. Ask it what it thinks about me and category theory. Then look at the references. It is mixing me up with other researchers.

- Undermind

  I think this is a very nice new addition to the search environent, but your free use is very limited. In the paid version you can easily identify articles that seem central to a field of research.

  Sign up and try your free search.

- Research Rabbit To be added.

15.2.2 More on Citation Management

As the web as grown as the principal compendium of scientific articles the community has had to deal with the facts that URLs (website addresses) change. How do we avoid dead links when searching for literature? At the moment, our best tool is to refer to the `doi`.

What does doi stand for?

Find the article with the doi: 10.1371/journal.pone.0152353

There are many tools for this, but a convenient look up is provided by Crossref.

If you keep your own reference database, you will often find that locating the doi is the easiest way to get that reference saved to your database.

The same problem that occurs for uniquely specifiying research articles can also occur for uniquely specifiying researchers. There is more than Britt Anderson out there. How would a reader know if I am the one who wrote a particular article?

What is an orcid?

After you know what one is. Get yourself one if you do **not** already have one.

15.2.3 The last thing you want to do ...

when managing references is to be cut and pasting them into your own home grown document. You want to use a piece of software that can easily keep the data updated and correct, and facilitate you easiliy being able to correctly cite and correctly format your references when you write. Again, there are many tools, but zotero has withstood the test of time and is a free offering well supported by academic libraries world wide.

Zotero is a piece of software, and for this section you ought ot download it and find a few articles on the web, in google scholar, or arxiv that you can install in your database to have a few test articles to work with. The support page will give you links to a quick start guide. There are many ways you can use zotero with tools you have used before (e.g. linking it to a word document), but in a moment we will explore a couple of ways to use it with the Quarto document writing system.

Since we don't want to have to manually format all our citations and references, and then reformat them when we are forced to by needing to resubmit to a new journal we want to rely on the computer to fix such things for us. One contemporary tool that can make that easier is csl. A *csl* file is a file that specifies all the necessary details about how references should be formatted for a particular citation style. Since you will most often be working with apa see if you can find and download the proper csl file https://www.zotero.org/styles, and then at least one other csl type so you can practice how easy it is to switch from one style to another.

15.3 Using References With Quatro and VS Code

Since you will, at least for this course, be writing Qmd files you may want to use Zotero for reference management and connect [it to Quarto](https://quarto.org/docs/visual-editor/technical.html#citations-from-zotero). You will find details under the citation section of this webpage. In

addition you will see how doi's, crossref, and other tools can be used to cite while you write in quarto. This workshop also provides details on citation management in Quarto as well as RStudio. See if you can write a minimal stub of an article and try some of the citation managment features in VS Code and Quarto.

15.4 Using References With Overleaf

VS Code is not your only option. Overleaf also use ".bib" files to hold references, though some of the other syntax is different. We have seen this before. Zotero will offer you the option to export references to a bib file which you can then use directly or if you need to submit a `.bib` file for a manuscript.

In addition you can link Overleaf and Zotero to make your writing easier.

Chapter 16

Large Language Models

16.1 When, why, whether?

Large language models are better for some tasks than others. They have become very good at writing the sort of simple, short programs that are common in experimental psychology. They can be very good at finding bugs or giving you a template to build on.

They can also be good, but you should be less confident, in acting as an expert conversation partner for you to bounce idea off of.

Whether to use them often depends on your goal. When you don't care how the result was achieved, *and* you have a way of verifying the output they can be great time savers. An example of this might be where you are having trouble figuring out why your formatting is off in a document. You just want it fixed and you will be able to tell by looking if it is fixed. Then let an LLM fix it.

If you want to learn something, then you may wish to *not* use an LLM. People who code a lot often find that there are unconventional choices made by LLMs. They may find overly complex solutions. They know this, because they can read the code and see what the LLM has done. If you don't know how to read the code yourself you may grow a mass of spaghetti that will cost you more time untangling than you thought you saved with the LLM. When you are beginning to code there is great value in learning the basic structures on your own, and then using that knowledge to bootstrap the quality of code your write later on with an LLM.

16.2 How to run

Wean yourself from the browser interface and the free versions of the mainline popular models. The free versions are often not the state of the art models the

companies have built. You are not getting the best answers. You end up in a constant state of cut and paste and that can introduce errors. It is harder to track your conversations over time and build on or continue chats or even change, in the middle of a conversation, which model you are using.

16.2.1 Pay up

To get the state of the art models you will need to pay, however, it can be difficult to choose which model and which plan. To help with this there are several aggregators that will funnel your requests to any of a number of models. You pay one fee to the aggregator and then the aggregator passes on some of the money to the host model. You can buy one set of credits that you can easily use with multiple vendors.

16.2.1.1 OpenRouter

Openrouter is the vendor that I currently use for this purpose. An account is free to create and use. You can only use free models and then you don't have to pay at all, but I recommend buying 10.00 of credit and then you can test all the new models as they come out.

To find the free models look for the "models" menu near the top right. After selecting it you can type "free" in the search bar and see the models that are free. The small clipboard icon after each models name will allow you to copy the official model name you will need to access it later with the tools described below.

16.2.2 Aider

Aider allows you to invoke the AI model from your terminal. You type `aider` at your terminal and off you go.

Before you begin though you will want to locate yourself in the directory where the project you are working on is located. If you are just playing, create a test directory and `cd` into that before beginning. Aider will, by default, ask to create a git repository to track its changes. Say yes. This allows you to more easily revert any of the changes aider makes. If you already have a git repo allow aider to commit to it. Aider will ask.

Since you are in the terminal your conversation appears in the terminal and commands to aider are preceded by a forwardslash (/). `/help` will get you a list of available things to do. Try starting with `/ask`. All the conversation occurs in your terminal and even the things that look like code do not get written anywhere. `/architect` will let you get aider's design ideas on a project. You can `/add` files and then ask aider to edit and fix them. For example, if you had a simple posner cuing task and now wanted to record reaction times in a csv file for each trial, you could ask aider to do that and it would, with your permission,

edit your code to make the changes. To control which model aider is using you use the `/model` command to then type or paste in your choice.

Having trouble getting your qmd file to behave exactly as you want? Let aider take a crack at it.

The usage guide for aider is found here.

16.3 Run locally

Using a LLM can mean different things. There is the training of the LLM and then there is the inference made by the LLM. The former often takes the large data centers that you hear about, but it can be done, for smaller size models, done on smaller set ups. If your training corpus is small it might be feasible on a single computer with a large GPU. However, that is not the usage that we will be addressing here.

If you want to make use of an already trained model for inference then you definitely can do that on a great many recent laptops. For these setups RAM is key, and a speedy CPU helps, but with advances made in model quantization most laptops can run something locally. That means no internet.

The appeal of this is learning and privacy. The data is yours. It doesn't leave and it is not trained on.

Depending on how much you want to get your hands dirty and learn the basic tooling, for which your python programming skills should be sufficient by the way, you can go with a graphical application that hides the details, or you can use something that you can control via the terminal. I will present both options.

16.3.1 LM Studio

LM Studio will give you the ability to download an app that you can then use to download all the billions of weights that make up a particular model. Start small if you have a lower spec'd laptop. After that you can converse via the app and your conversations are private and local. You can also upload up to five sources (e.g. pdfs) that will then be used to aid the model in retrieving answers pertinent to that material (a process called RAG for retrieval augmented generation).

16.3.2 Ollama

Ollama is an online repository of a number of very powerful models and provides the tooling for you to be able to run them locally. The quickstart guide provides your best starting point.

With this software installed you can run larger models locally. Embed large collections of documents and run special inquiries and conversations with tailored

prompts for particular collections. It is worth a look, but it is beyond what we will be doing in our course here.

https://www.youtube.com/watch?v=TzY2lzrlPuI

16.3.3 Using LLMs for Dynamic Research

With a model running locally you could use this in your experimental pipeline to take in user input and return model output.

What are the mechanics we need for this? We need our model to accept input and generate output and we need a method to transfer the elements of the conversation back and forth. The first stage is to think of the components we have used so far and how we might stitch them together for this task.

It is also a useful transitional step to work incrementally. First, maybe try to set up an interface to allow someone to talk to the model via a web interface. Then you might be able to figure out how to use GET and POST from PHP to communicate between two web servers. There are definitely better ways to do that, but it is a fairly direct starting point. If you want an additional challenge perhaps try to use the tool: curl.

16.3.4 Why Might You Want To Do This?

Because you would like your paper to be published in the world's premier scientific journal *Science*.

Durably reducing conspiracy beliefs through dialogues with AI

https://www.youtube.com/watch?v=qD1fnYDTbHM

This LinkedIn post talks about ChatGPT integration. What would you have to change to get it working with your local server?

Chapter 17

In class practice exercise

17.1 Prerequisites

- OpenRouter account created (free at openrouter.ai)
- Aider installed on your system
- Basic familiarity with the terminal/command line

17.2 Get Your OpenRouter API Key

1. Log in to OpenRouter
2. Navigate to "Keys" in the top menu
3. Click "Create Key"
4. Copy your API key (you'll need it in a moment)

17.3 Set Up Your Environment

In your terminal, set your OpenRouter API key as an environment variable:

On Mac/Linux:

```
export OPENROUTER_API_KEY='your-key-here'
```

On Windows (PowerShell):

```
$env:OPENROUTER_API_KEY='your-key-here'
```

17.4 Create a Working Directory

```
mkdir ~/aider_tutorial
cd ~/aider_tutorial
```

17.5 Copy the Buggy Pong Game

Copy the `pong_buggy.py` file from the course materials into your working directory.

17.6 Launch Aider

In your terminal, from your working directory, type:

```
aider
```

Aider will ask if you want to create a git repository. Type `y` and press Enter. This allows you to track changes and revert if needed.

17.7 Select a Free Model

At the aider prompt, type:

```
/model meta-llama/llama-3.2-3b-instruct:free
```

This connects you to a free model via OpenRouter. You should see confirmation that the model has been set.

17.8 Add the File to Aider's Context

```
/add pong_buggy.py
```

This tells aider which file you want to work with.

17.9 Ask Aider to Identify Problems

Type the following at the aider prompt:

```
/ask Can you review this pong game code and identify any bugs or issues that would prev
```

Important: Read the response carefully. Does it identify the bugs? Does it explain them clearly?

17.10 Request Fixes

Now type:

```
Fix the bugs you identified in the pong game code.
```

Aider will show you the proposed changes. If you agree, type `y` to accept the changes.

17.11 Verify the Fixes

Exit aider by typing `/exit`.

Run the pong game:

```
python pong_buggy.py
```

Test the game: - Use W/S keys to move the left paddle - Use Up/Down arrow keys to move the right paddle - Does the ball bounce correctly off both paddles? - Does the score update correctly when the ball goes past each paddle?

17.12 Review the Git History

```
git log --oneline
```

You should see commits made by aider showing what was changed.

To see the actual changes:

```
git diff HEAD~1
```

17.13 The Bugs (Reference - Don't Peek!)

The file contains two bugs:

1. **Scoring bug:** When the ball goes past the right paddle, `score_a` is incremented instead of `score_b`
2. **Collision bug:** The x-coordinate check for paddle_b collision is wrong (checks `> 300` instead of `> 340`)

17.14 Key Takeaways

- LLMs can be very effective at identifying logical errors in code
- **Always verify the output** - just because the LLM suggests it doesn't mean it's correct
- Using version control (git) with aider lets you safely experiment and revert changes
- Free models can handle many programming tasks adequately
- The `/ask` command is useful for exploration; direct commands make actual changes

17.15 Next Steps

For homework, you'll use these same skills to build a lexical decision experiment from scratch!

Chapter 18

Summary

In summary, I have not gotten to writing a summary yet.

Appendix A

Mac Related Instructions

A.1 Homebrew

Homebrew installation instructions

A.2 Git

Install Mac

A.3 VS Code

Install Mac

A.4 Texlive

Get it via Homebrew with `brew install texlive` then export to pdf via quarto works well.

A.5 UV

This is a new tool that will take care of installing python version, python libraries, and initiating virtual environments. Many operating systems come with a version of python already installed. If you install additional versions to the global workspace you can get conflicts that can be very frustrating and challenging to fix.

First, get uv.

Second, install a version of python.

Third, practice setting up a virtual environment.

Fourth, install some packages.

A.6 R

Install instructions

A.7 Quarto

Getting started

Appendix B

Windows Specific Advice

B.1 General

Go slow.

Read, pause, reflect, and then, and only then, type and hit enter.

B.2 Context

Windows is a very locked down system. Developing applications for windows using only windows tools can be limiting. You don't know what you will want to do in the future. By using a linux style system with linux inflected tools you will be learning general methods that will broaden what you can do in the future, but living in two worlds can be tricky and you will need to keep straight when you are in windows proper and when you are in the subsystem for linux.

B.3 Windows Subsystem for Linux

This is a relatively new feature for the Windows operating system that lets you install a linux virtual machine inside your current Windows distribution. You can thus "use" linux while still running windows. You can use it just for terminal commands or you can run some visual applications as well.

B.3.1 Make sure you are on Windows 10 or 11 and that you are updated.

There will be two steps to this process. First you will install windows subsystem for linux, usually just called WSL. This will be easier if you have the windows power shell available.

Once you have installed WSL you will then install a linux distribution. Since ubuntu is one of the most common and best documented of the linux distributions that is the distribution I suggest you start with. You will use power shell to get wsl and ubuntu installed. After having done that you will primarily use the linux terminal that you have installed. This can be launched via you applications. Just look for Ubuntu.

B.3.2 Then follow the instructions per microsoft.

B.4 VSCode

This is an integrated development environment for numeous coding languages including the three file types that we will focus on: .qmd, .py, and .r files.

Downloading it and having it play nice with your linux environment requires special care. Make sure you follow instructions specifically related to wsl. Usually you will install the Windows VS Code application, but when you go to "launch" it for files that will be stored in your linux land you will use your *linux* terminal with the command `code .`. Don't forget the period. That should start it from your home directory in linux land.

Install

B.5 Git

If you are using WSL then you probably already have it. Enter you WSL terminal and try `git --version`. If you see an answer you have it. If you don't want to use WSL you can install it for Windows directly.

B.6 Python

First, get uv.

Optional, install a version of python. It may be better *not* to do this. You can get the particular version of python you need at the time you create your virtual environment in step two below.

Second, practice setting up a virtual environment.

If you are in your linux terminal you will use terminal commands such `cd` to navigate to a particular directory and then I recommend making a test directory until you are confident in what you are doing. For example, `cd ~` takes you to your home directory; `mkdir test` creates a directory called "test"; `cd ./test` moves you into your test directory, and finally, `uv venv -p 3.10` will install a virtual environment in the "test" directory (called `.venv` – you would have to `ls -a` to see that). You will then activate this environment with `source .venv/bin/activate` to make it live, and `deactivate` when you are done.

Third, install some packages. *After* having created the virtual environment and activated it.

Appendix C

Python

Appendix D

Virtual Environments

The advantage of a virtual environment is the ability to isolate libraries you may install for a particular project from interfering with other projects or previously installed libraries. The disadvantage is that they do involve some additional steps and complexity and you may end up installing multiple versions of the same library, which can be an issue if hard disk space on your computer is limited.

D.1 Methods

There is more than one tool for creating a python virutal environment, but one of them is built into the standard python installation: *venv*, and so for simplicity this is the one that I recommend starting with.

D.1.1 Creating and Activating the Venv

If you are in a console you can simply run these commands in the terminal.

```
python -m venv <dir-where-you-want-venv>
cd <dir-where-you-want-venv>
source ./bin/activate
```

> **Note**
>
> Depending on the specifics of your operating system and python installation you may find you need to use `python3` and `pip3` commands instead of plain `python` and `pip`.

If you are using a Windows device you will have to find your equivalent of a linux terminal. This can be using WSL or something like MinGW.

Once you have activated your `venv` you will see a change in the terminal display that indicates the `venv` is active. Now when you run things like `pip install <something>` it will install that library to a sandbox that is only accessible when using python from within that virtual environment. You should note that this notion of a sandbox or virtual environment is also available for some other programming languages.

To deactivate the environment you run ,

```
deactivate
```

from within the venv. Look for the change in the terminal display. Note how activating the environment shows up in the terminal prompt.

```
britt@arts-220486 ~ % source p390venv/bin/activate
(p390venv) britt@arts-220486 ~ % deactivate
britt@arts-220486 ~ %
```

D.1.2 The Terminal is All You Need

If you are a minimalist or doing something simple you can invoke the python interpreter in your venv and complete your task there with no other tools, and using only the libraries required for your particular, limited task. However, if you use some companion tooling things may get more complicated and trickier to debug.

D.2 Visual Studio Code

VS Code tries to make using a venv easier, but this also means there is some indirection that may make it harder to puzzle out errors. The basic work flow is to make sure that VS code is working in the folder you want your venv to be in. If not, use the `open folder` menu of VS Code to do so. Then, you can use the `Command Palette` (found under the *View* menu tab) to locate the `Python Create Environment` command. If you have never created an environment before it will do so. This can take some time so be patient and make sure this process completes. If there was a virtual environment previously constructed VS Code will probably be able to figure that out and ask you if you want to re-use it. Usually you will, but if not you can select to destroy it and create a new one. You will probably also need to affilate a particular python version to association to your environment. VS Code can often make a reasonable suggestion and you can also use the command `Python Select Interpreter` if necessary.

After you have done that you will need to move into the affiliated terminal window of your VS Code environment to install the necessary libraries. You will need to be deliberate and make sure that you are in the correct terminal. VS Code may have several of these open for different purposes if you are working on a complicated project.

Note that while it may be convenient to use the VS Code terminals you do not have to. If you have a terminal program on your computer accessible to you outside of VS Code you can use VS Code as the editor and then move to your other terminal for activating and implementing and compiling the python project.

D.2.1 Quarto Complications

When you embed a python code block in your .qmd document you will want to compile it. Quarto is assembling a document. It will need more than just the libraries necessary for compiling the python code. It also needs the library for the textual and graphical representation of the output that you will insert into your document. Specifically, it needs the `jupyter` python library. This means that whatever the python is doing, even if no additional libraries are needed, you will have to have done a `pip install jupyter` in your venv *and* you will need to have the venv active for the location where the qmd is stored and compiled.

It is worth knowing that although VS Code offers the convenience of buttons to invoke the quarto process it is *not* necessary. You can invoke all the quarto commands through a terminal directly. For instance `quarto preview <file-name>.qmd` will try to compile your file and will also usually launch your browser so you can see the html result. It will then watch the source file and update the html whenever it detects you have changed the source. This can be quite convenient for editing.